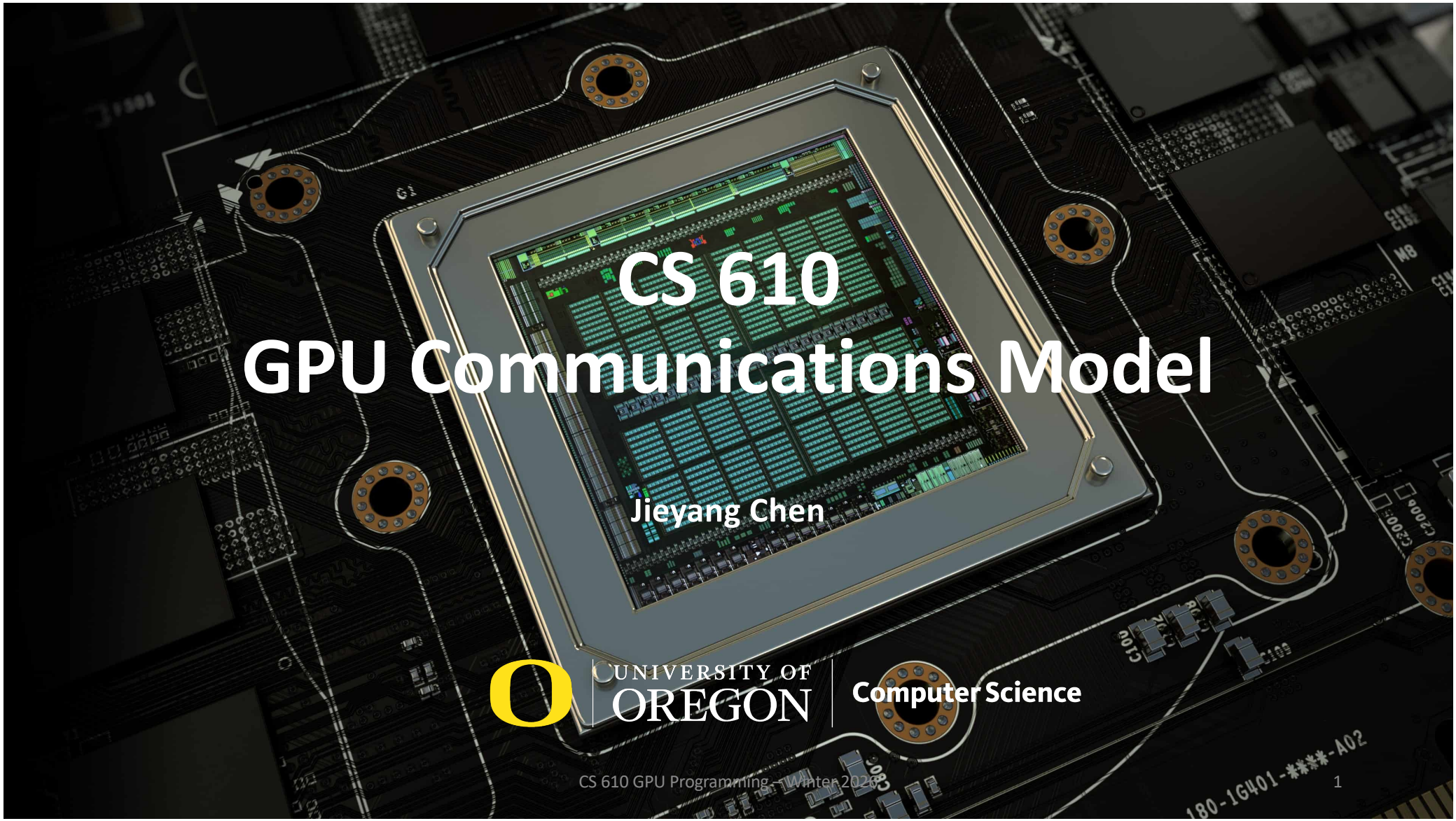


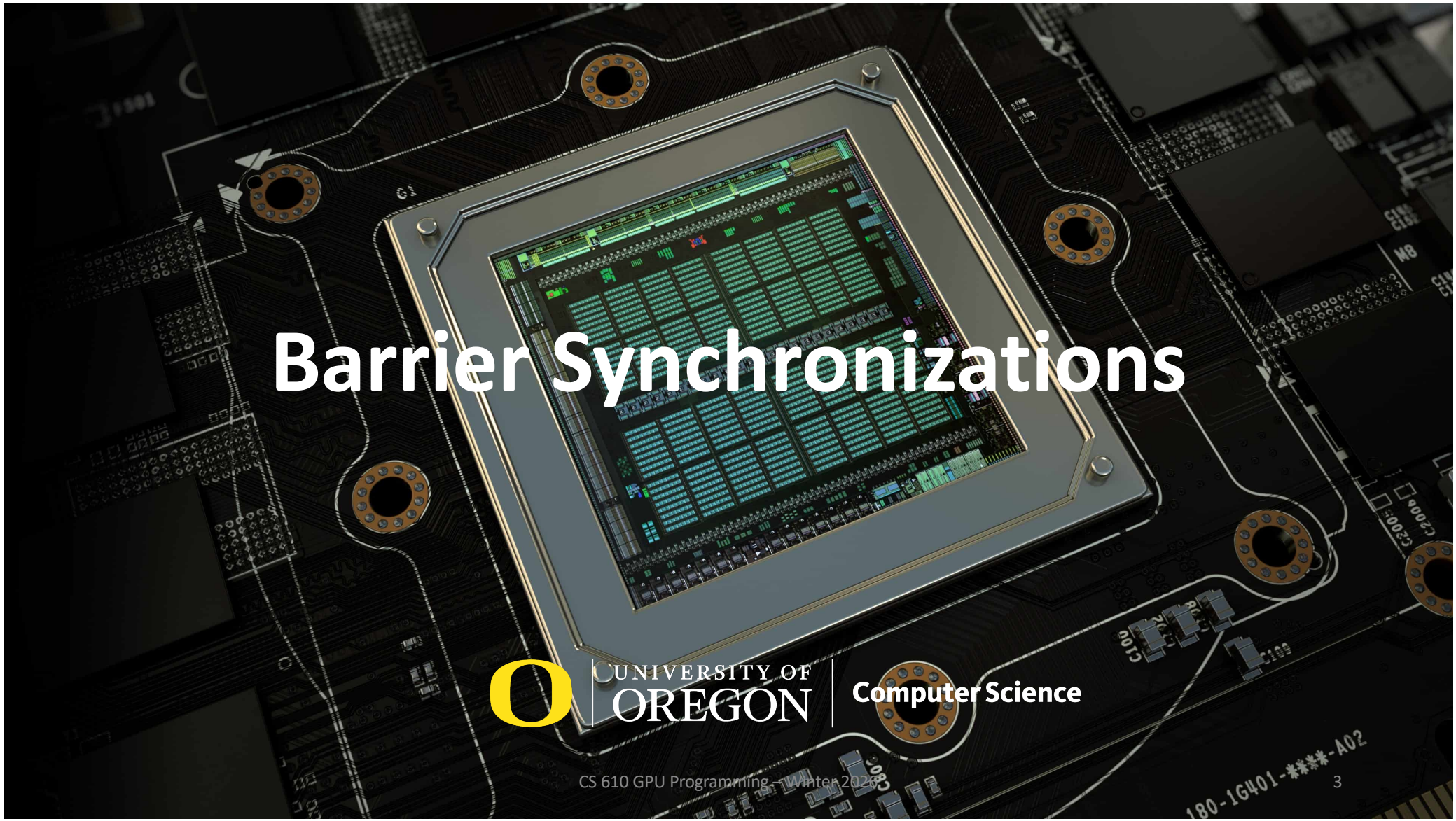
CS 610
GPU Communications Model

Jieyang Chen

UNIVERSITY OF OREGON | Computer Science

Communication in parallel computing

- Communication does not only mean data transmission
- Communication includes all kinds of interactions between threads
  1. **Enforcing executing timing/ordering**: barrier synchronization, memory barrier
  2. **Exchanging/sharing data:** data transfer, common memory space
  3. **Enforcing exclusive resource access**: atomic operations
- If a parallel algorithm does NOT require any communication, it is called Communication-avoiding Algorithm

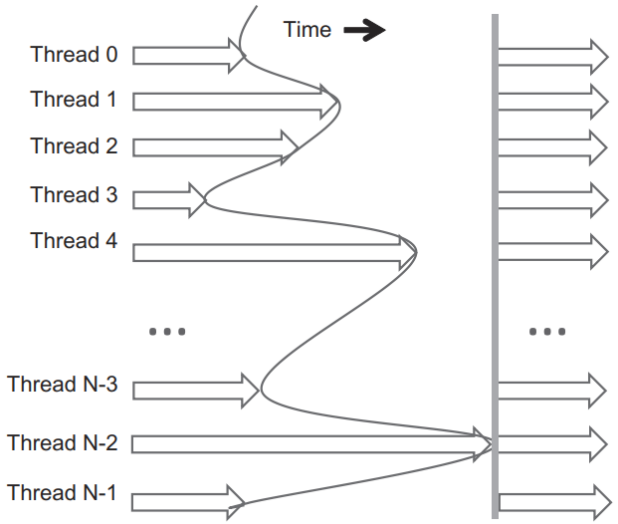


Barrier Synchronizations

UNIVERSITY OF OREGON | Computer Science

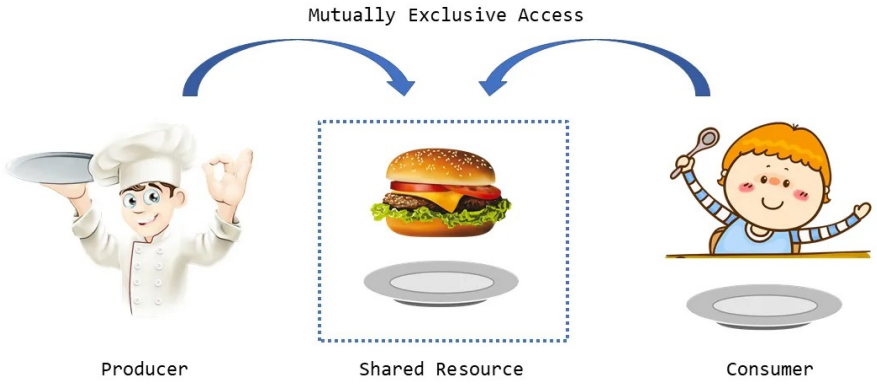
Barrier Synchronization

- Most fundamental type of communication
- Threads communicate each others' status to ensure all current works have completed by all threads before moving on to the next task
- Enforcing every thread's execution timing



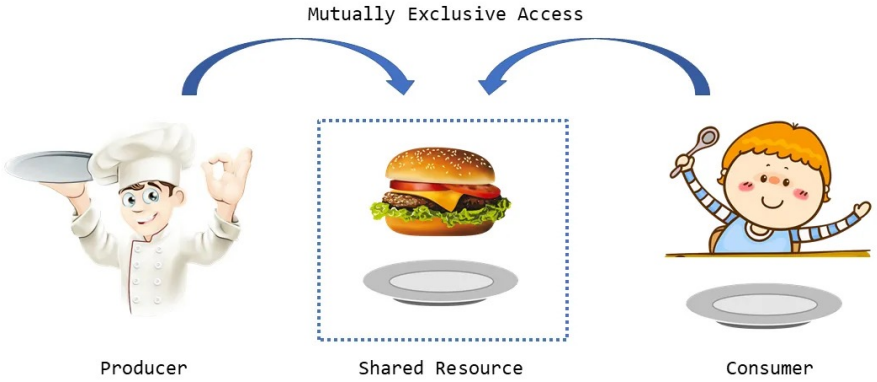
Consumer-Producer Problem

- A classic concurrency problem
- Producer threads produce data and consumer threads consume data
- They share data using a common data buffer
- The problem arises when producer and the consumer operating at different rates.

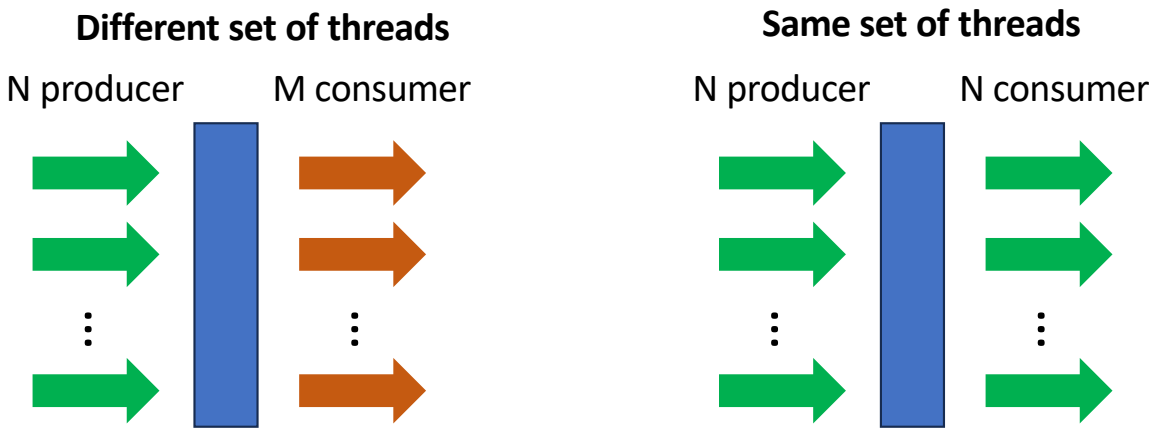


Consumer-Producer Problem

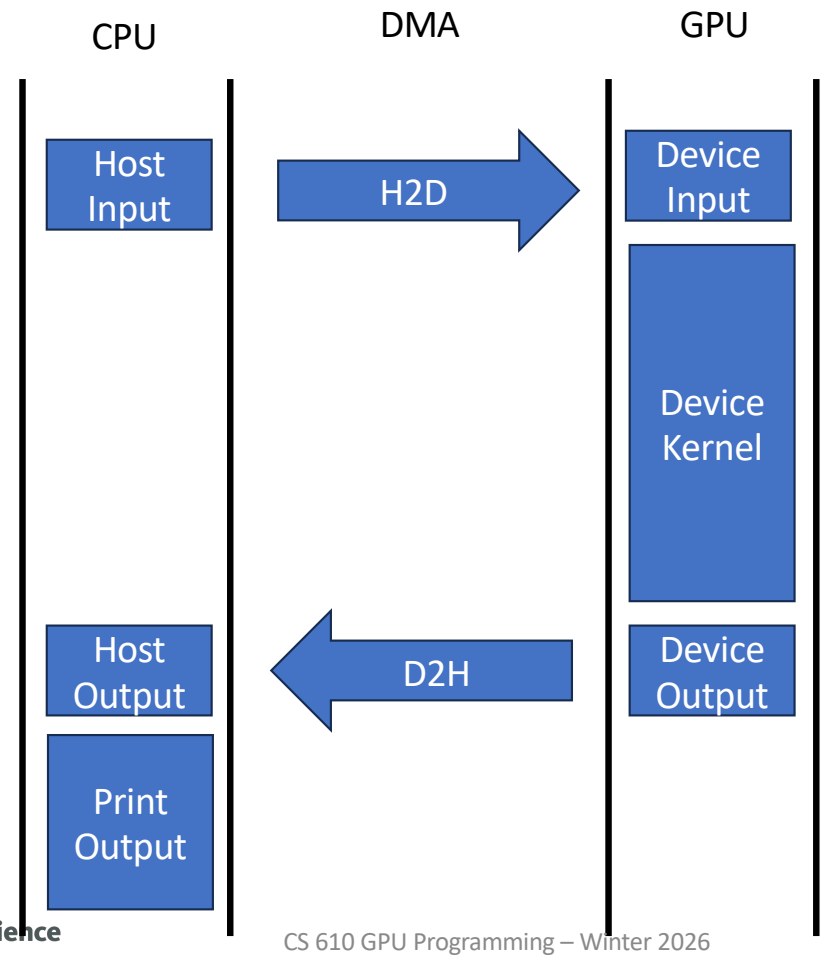
- Producer need to wait for existing data to be consumed before producing more data
- Consumer can only consume data if data is produced by the producer.
- They cannot access the data buffer at the same time because of data race.
- Barrier synchronization is a common solution!



Consumer-Producer Problem



Example

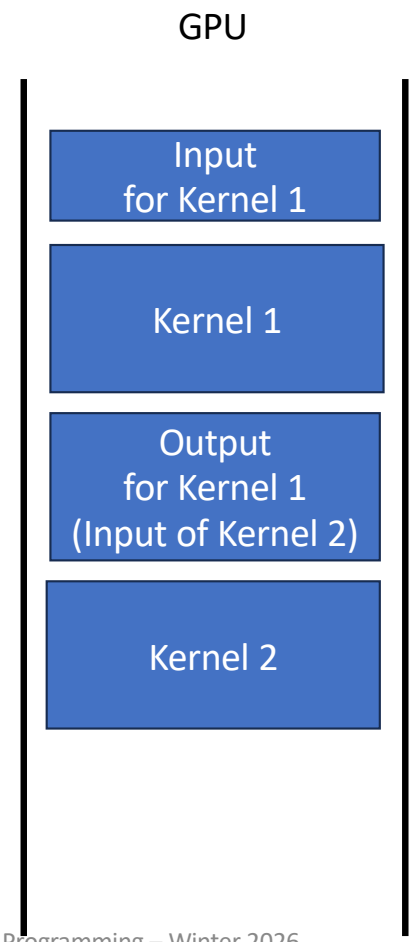


Producer: H2D-DMA
Consumer: GPU threads
Common buffer: Device input
All GPU threads needs wait for H2D before consume data

Producer: GPU threads
Consumer: D2H-DMA
Common buffer: D2H-DMA needs to wait for all GPU threads finish before starting to transfer the data to host

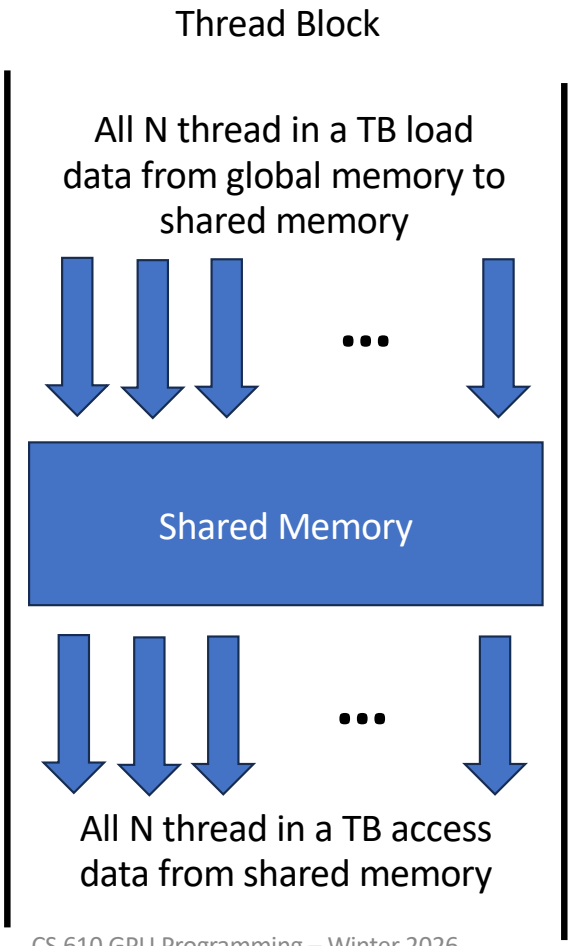
Producer: D2H-DMA
Consumer: CPU
Common buffer: Host output
CPU needs to wait for D2H before print output

Example



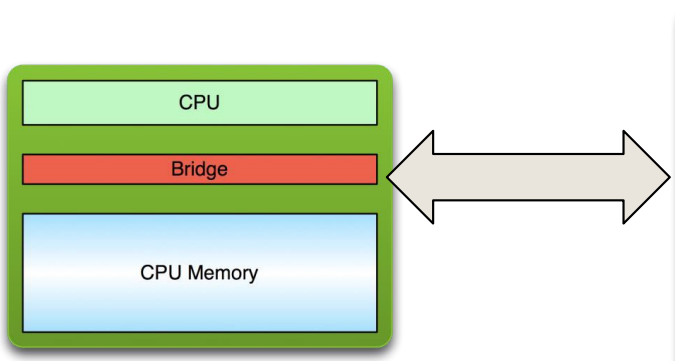
Producer: Kernel 1 (N threads)
Consumer: Kernel 2 (M threads)
Common buffer: Output of Kernel 1
All GPU threads in kernel 2 needs wait for all GPU threads in kernel 2

Example

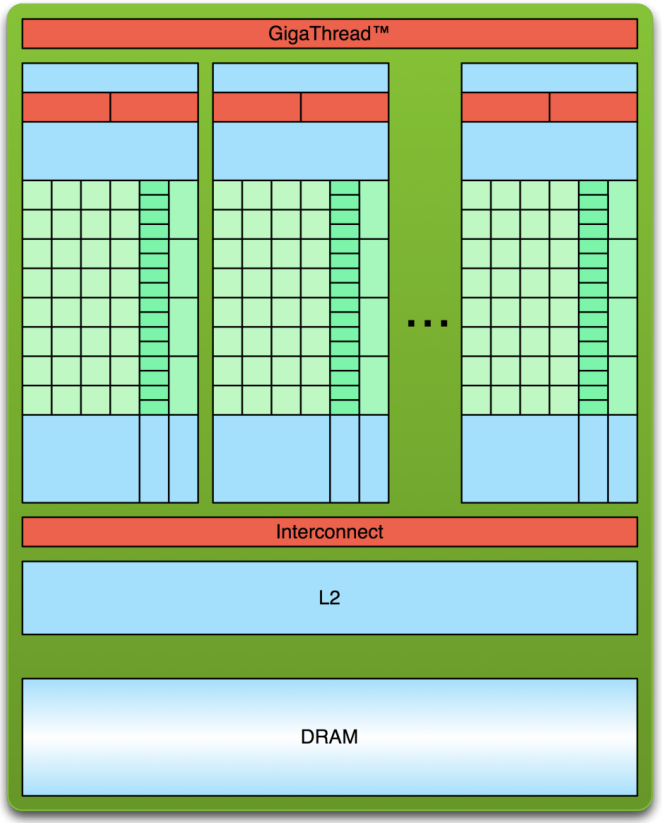


Producer: N thread in a TB
Consumer: N thread in a TB
Common buffer: Shared memory
All N threads in a TB needs wait for loading complete before access the data

Parallelisms



CPU/GPU Processor



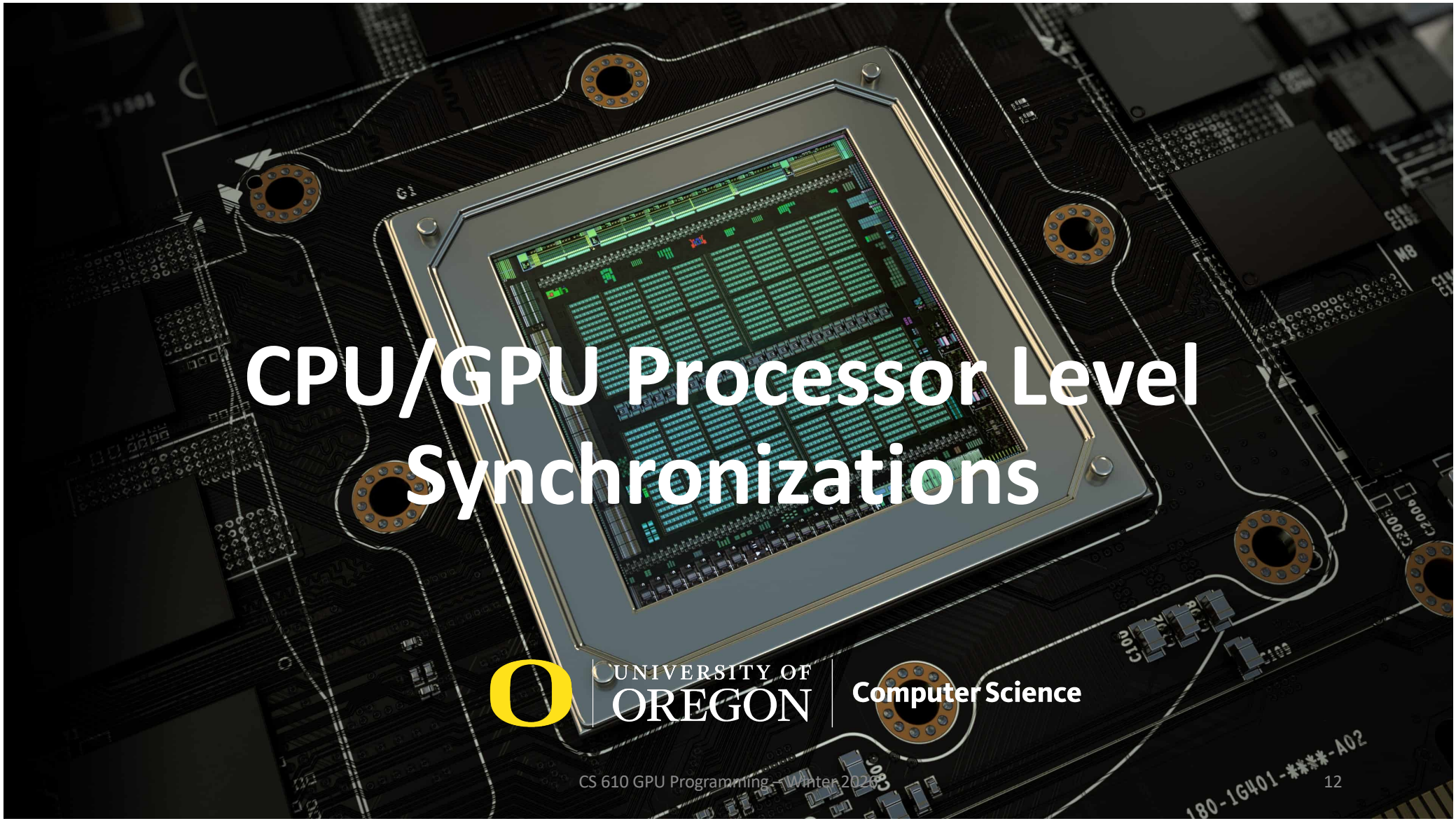
thread
(sub-warp)

warp

Thread Block

Grid

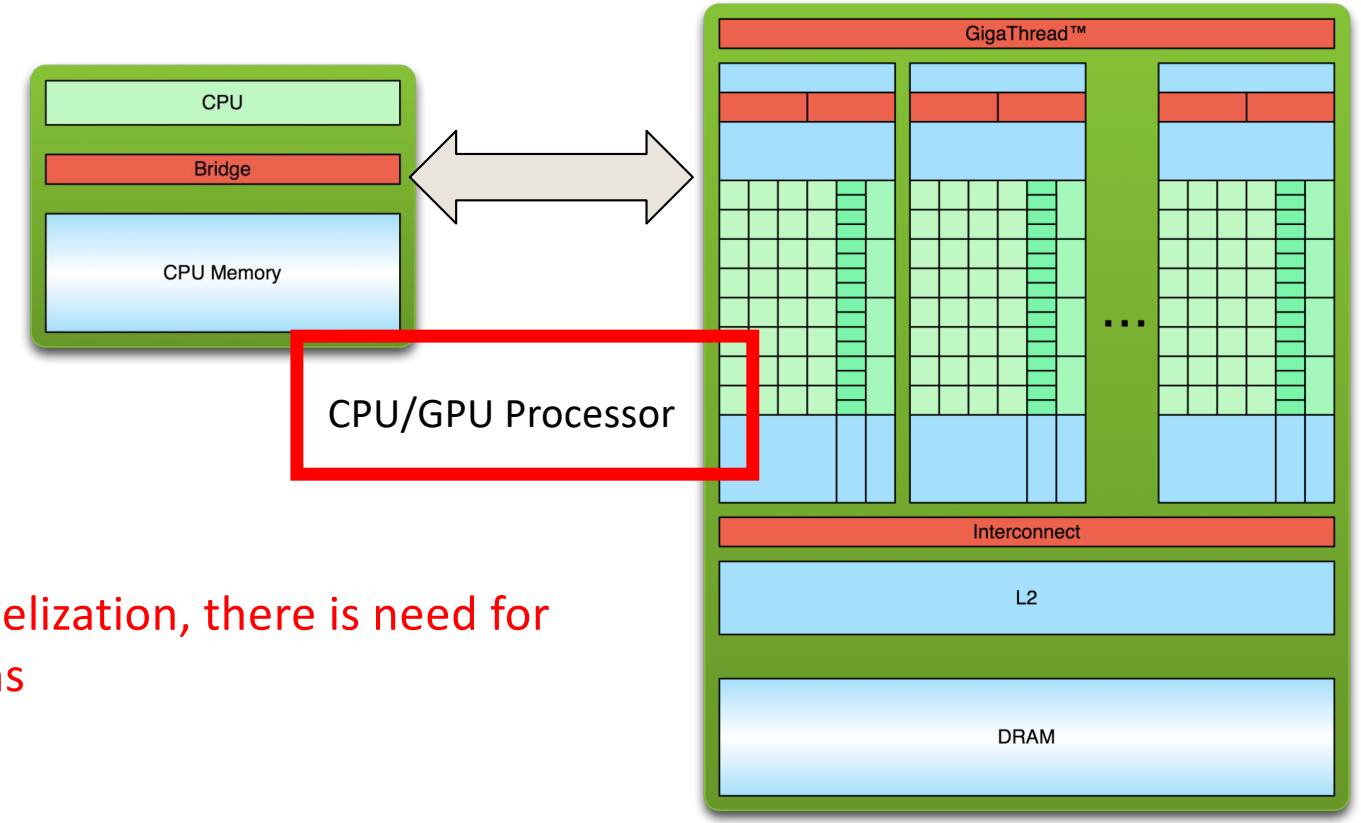
If there is parallelization, there is need for synchronizations



CPU/GPU Processor Level Synchronizations

UNIVERSITY OF OREGON | Computer Science

Parallelisms



If there is parallelization, there is need for synchronizations

CPU/GPU Processor Level Synchronizations

- Synchronize the tasks on CPU and GPU
- Enforcing the order of
  - CPU function and GPU kernel
  - GPU kernel and CPU-GPU data transfer
  - CPU function and GPU-CPU data transfer

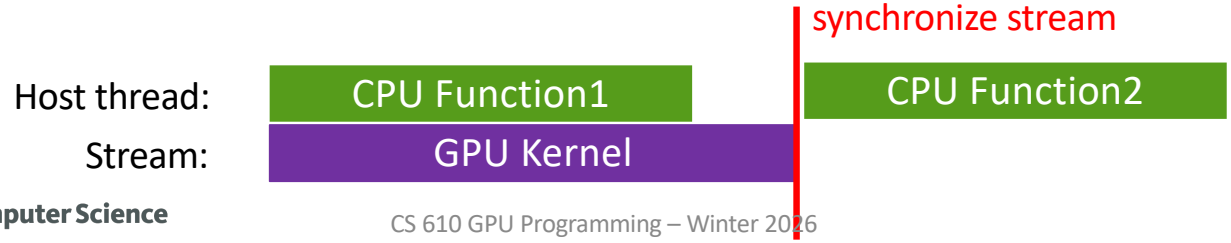
Synchronize Streams

- Wait for all tasks in a stream

cudaStreamSynchronize(stream)

Examples

```
Kernel<<<16, 256, 0, stream>>>(…parameters…);
CPU_Function1(…parameters...);
cudaStreamSynchronize(stream);
CPU_Function2(…parameters...);
```



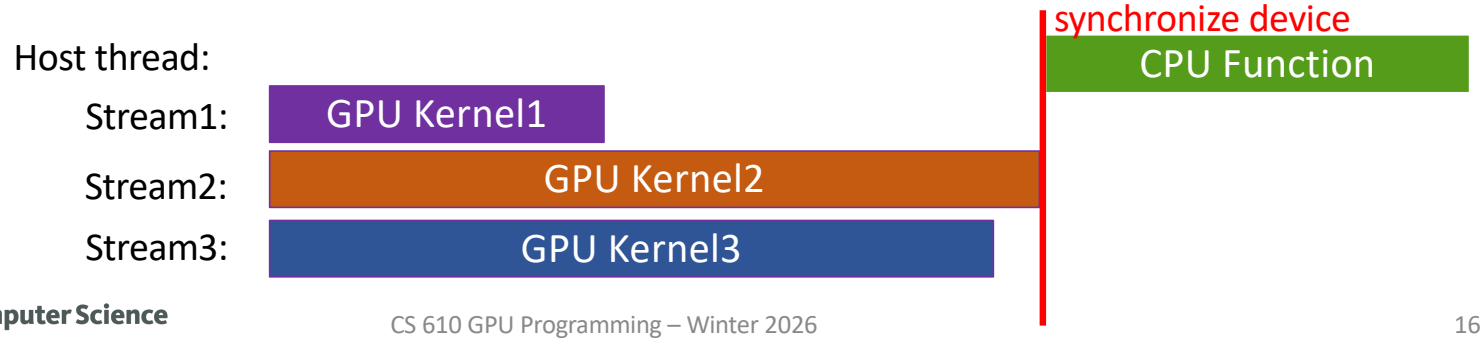
Synchronize Device

- Wait for all tasks on a device (all streams)

cudaDeviceSynchronize()

Examples

```
Kernel1<<<16, 256, 0, stream1>>>(…parameters…);
Kernel2<<<16, 256, 0, stream2>>>(…parameters…);
Kernel3<<<16, 256, 0, stream3>>>(…parameters…);
cudaDeviceSynchronize();
CPU_Function(…parameters...);
```



Synchronize CUDA Events

cudaEventSynchronize(event);

Wait for all operations preceding the event

Examples

```
Kernel1<<<16, 256, 0, stream1>>>(…parameters…);
cudaEventRecord(event1, stream1);
Kernel2<<<16, 256, 0, stream1>>>(…parameters…);
cudaEventSynchronize(event1)
```

Add an event in between kernel1 and kernel2
Wait for kernel1 only

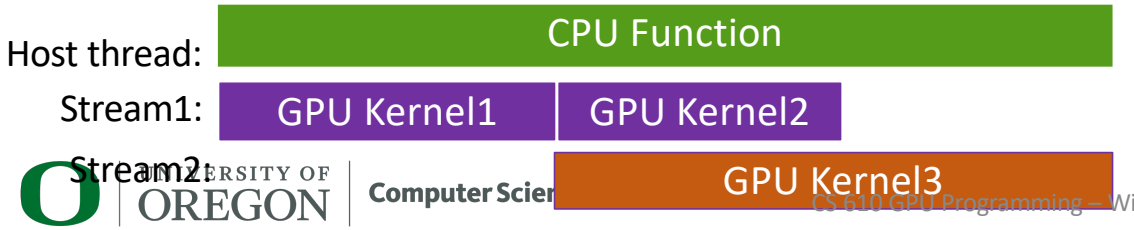
Synchronize CUDA Events with stream

cudaStreamWaitEvent(stream2, my_event);

- Makes all the commands added later to the given stream delay their execution until the given event has complete. The event can be on a different stream
- This function does not block the CPU threads

Examples

```
cudaEvent_t my_event;
cudaEventCreate(& my_event);
Kernel1<<<16, 256, 0, stream1>>>(…parameters…);
cudaEventRecord(my_event, stream1);
Kernel2<<<16, 256, 0, stream1>>>(…parameters…);
cudaStreamWaitEvent(stream2, my_event);
Kernel3<<<16, 256, 0, stream2>>>(…parameters…);
CPU_Function(…parameters…)
```



Delay kernel3 until my_event is completed

Explicit Synchronization

- **Synchronize everything**
  - **cudaDeviceSynchronize ()**
  - **Blocks host until all issued CUDA calls are complete**

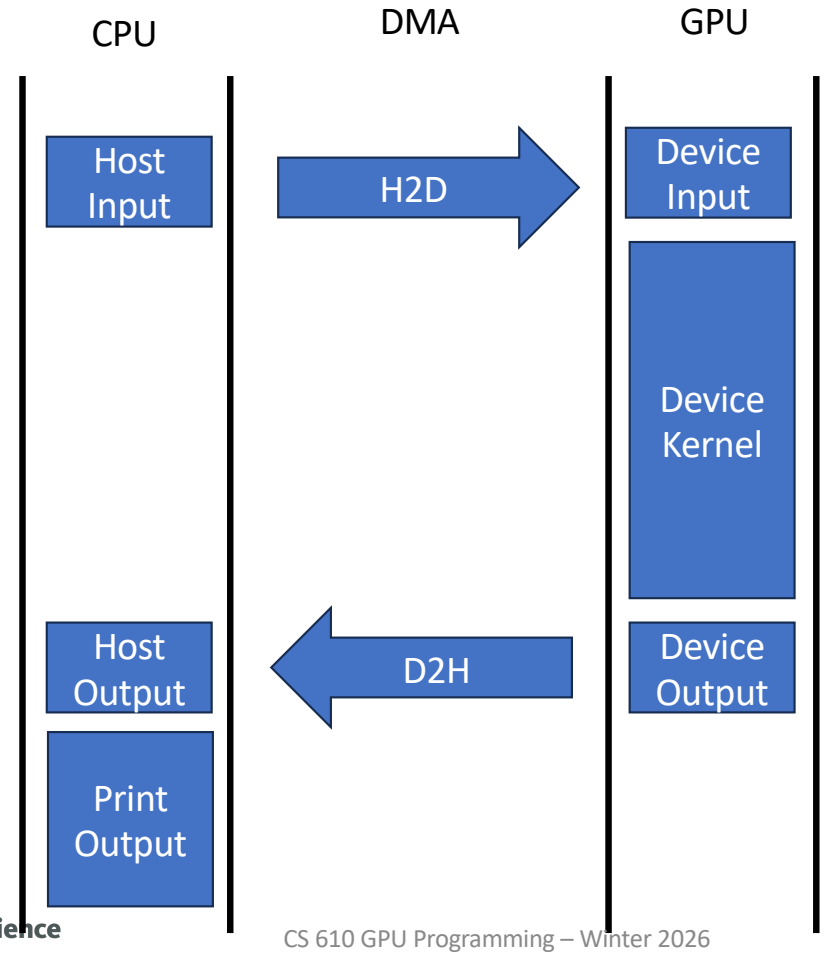
- **Synchronize w.r.t. a specific stream**
  - **cudaStreamSynchronize (** streamid **)**
  - **Blocks host until all CUDA calls in streamid are complete**

- **Synchronize using Events**
  - **Create specific 'Events', within streams, to use for synchronization**
  - **cudaEventRecord (** event, streamid **)**
  - **cudaEventSynchronize (** event **)**
  - **cudaStreamWaitEvent (** stream, event **)**
  - **cudaEventQuery (** event **)**

Implicit Synchronization

- These operations implicitly synchronize all other CUDA operations
  - Page-locked memory allocation
    - cudaMallocHost
    - cudaHostAlloc
  - Device memory allocation
    - cudaMalloc
  - Non-Async version of memory operations
    - cudaMemcpy* (no Async suffix)
    - cudaMemset* (no Async suffix)
  - Change to L1/shared memory configuration
    - cudaDeviceSetCacheConfig

Where are the sync?

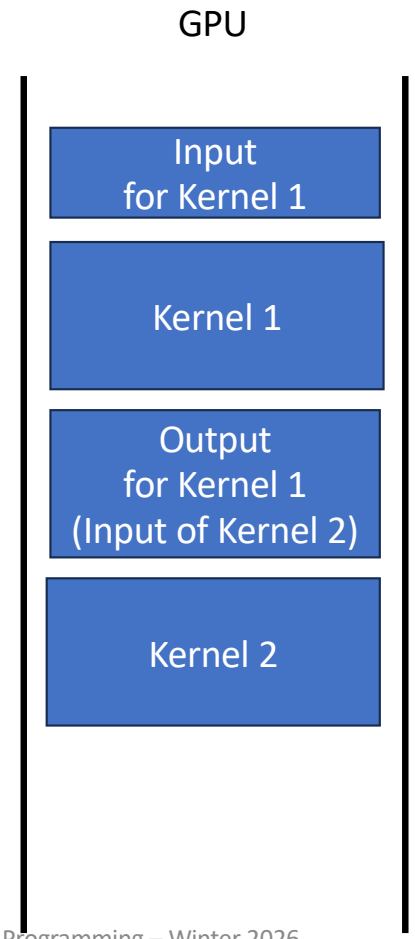


Producer: H2D-DMA
Consumer: GPU threads
Common buffer: Device input
All GPU threads needs wait for H2D before consume data

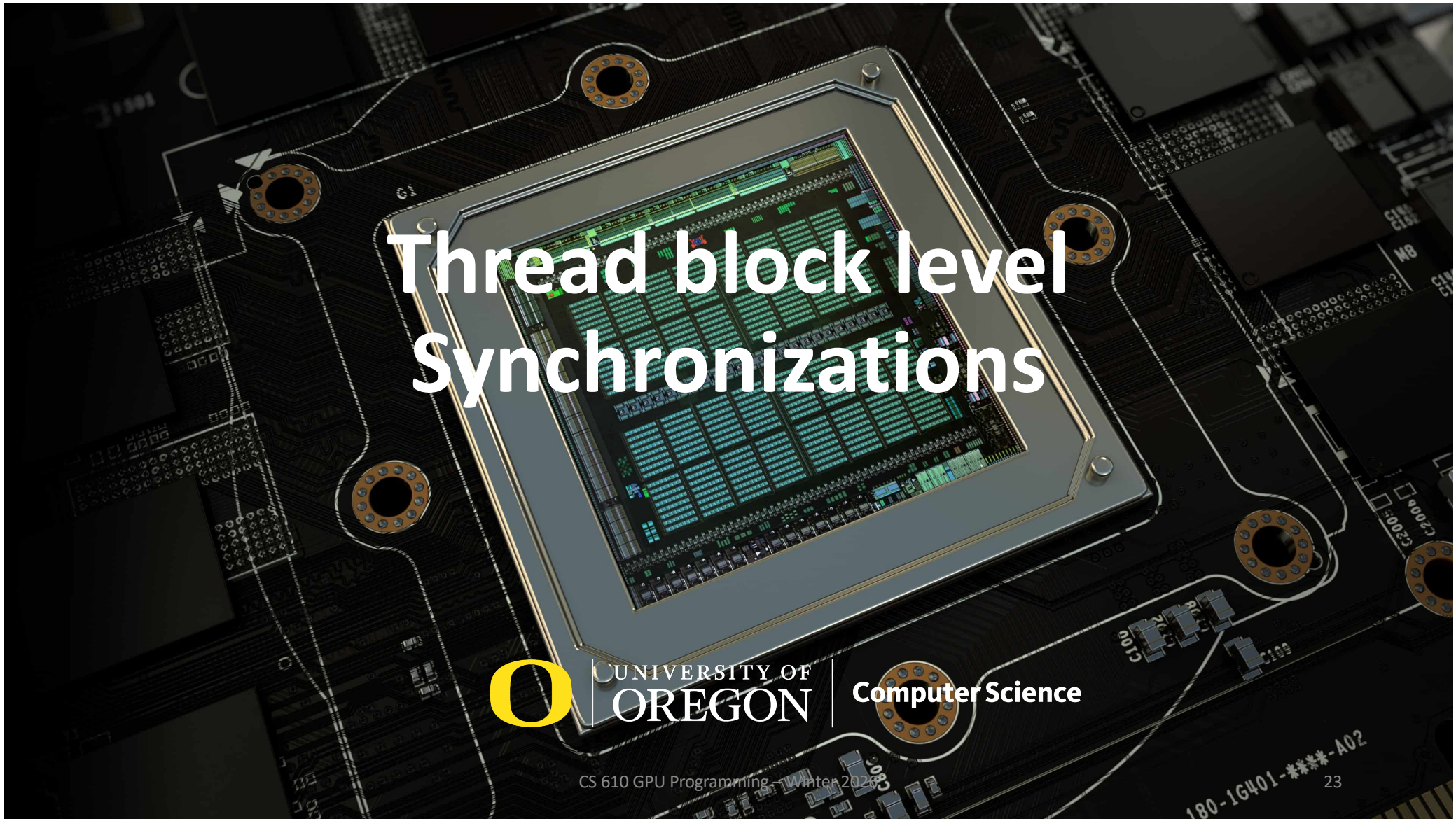
Producer: D2H-DMA
Consumer: CPU
Common buffer: Host output
CPU needs to wait for D2H before print output

Producer: GPU threads
Consumer: D2H-DMA
Common buffer: D2H-DMA needs to wait for all GPU threads finish before starting to transfer the data to host

Where are the sync?



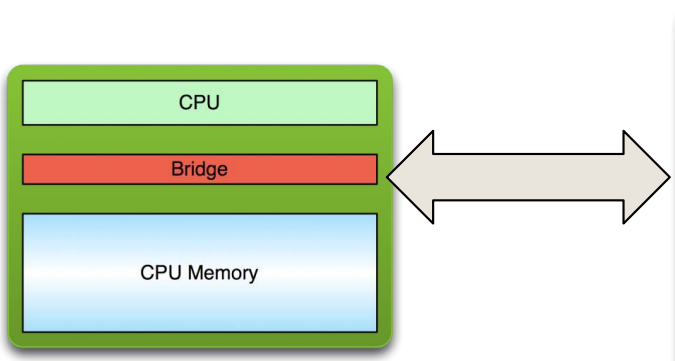
Producer: Kernel 1 (N threads)
Consumer: Kernel 2 (M threads)
Common buffer: Output of Kernel 1
All GPU threads in kernel 2 needs wait for all GPU threads in kernel 2



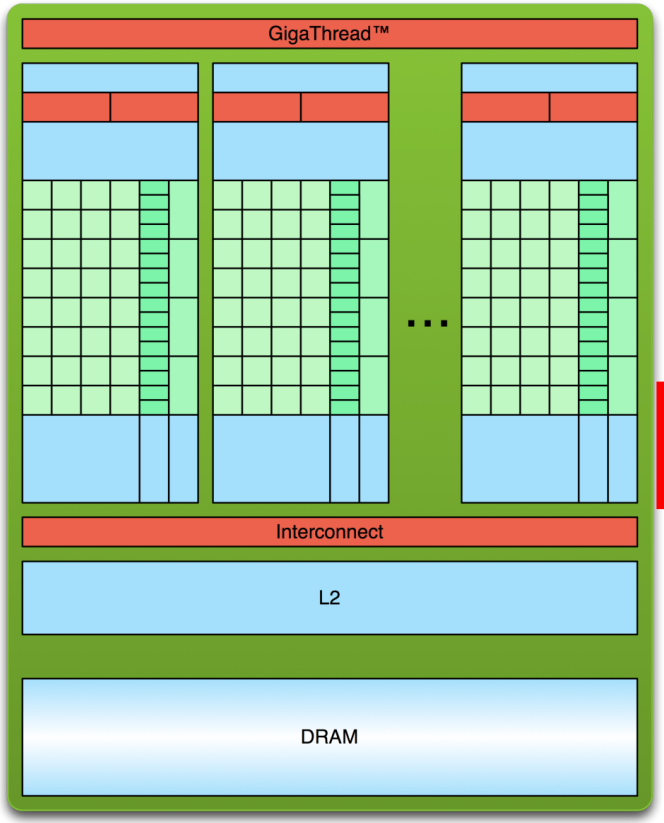
Thread block level Synchronizations

UNIVERSITY OF OREGON | Computer Science

Parallelisms



CPU/GPU Processor



thread
(sub-warp)

warp

Thread Block

Grid

If there is parallelization, there is need for synchronizations

Thread block level synchronizations

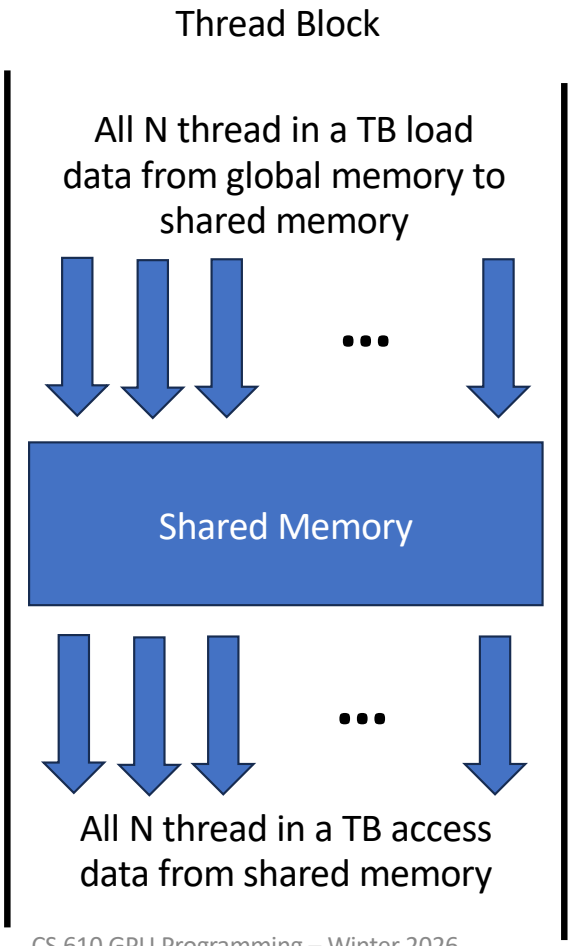
- Synchronize all thread in a thread block
- Enforcing all instructions of all threads before synchronization are finished
- Enable correct data sharing between threads in thread block

Thread block sync

__syncthreads()

- Hold all threads calling this function until every thread in the same block calls the function.
- Ensures that all threads in a block have completed a phase of their execution before any of them can move on to the next phase.

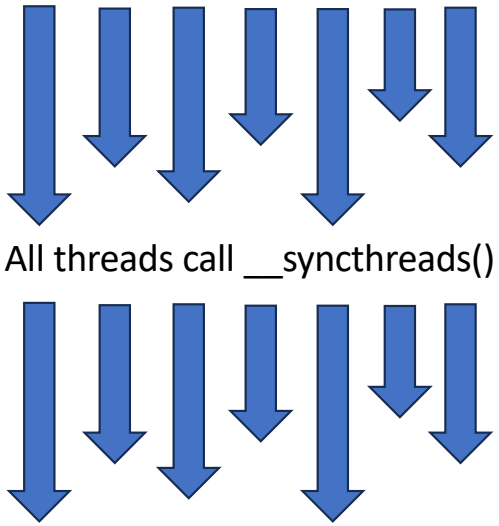
Where to put __syncthreads() ?



Producer: N thread in a TB
Consumer: N thread in a TB
Common buffer: Shared memory
All N threads in a TB needs wait for loading complete before access the data

Be careful when sync a thread block

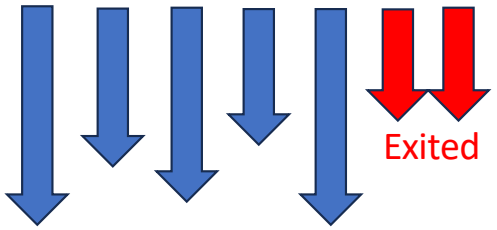
All threads in a TB need to call __syncthreads() before they can move on



Be careful when sync a thread block

All threads in a TB need to call __syncthreads() before they can move on

```
If (threadIdx.x < 2) {
   return;
}
.....
__syncthreads()
```



They will never call __syncthreads()

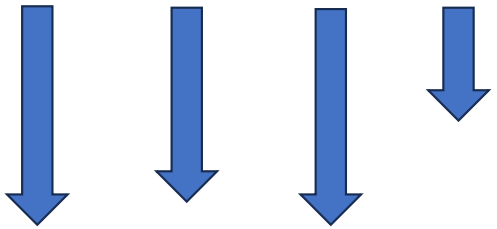
They will wait at __syncthreads() forever

Be careful when sync a thread block

All threads in a TB need to call __syncthreads() before they can move on

```
If (threadIdx.x % 2 == 0) {
    ……
    __syncthreads()
} else {
    …….
    __syncthreads()
}
```

Divergence



threadIdx.x % 2 == 0

They will wait at __syncthreads() forever

threadIdx.x % 2 != 0

They will never enter __syncthreads()

Deadlock!

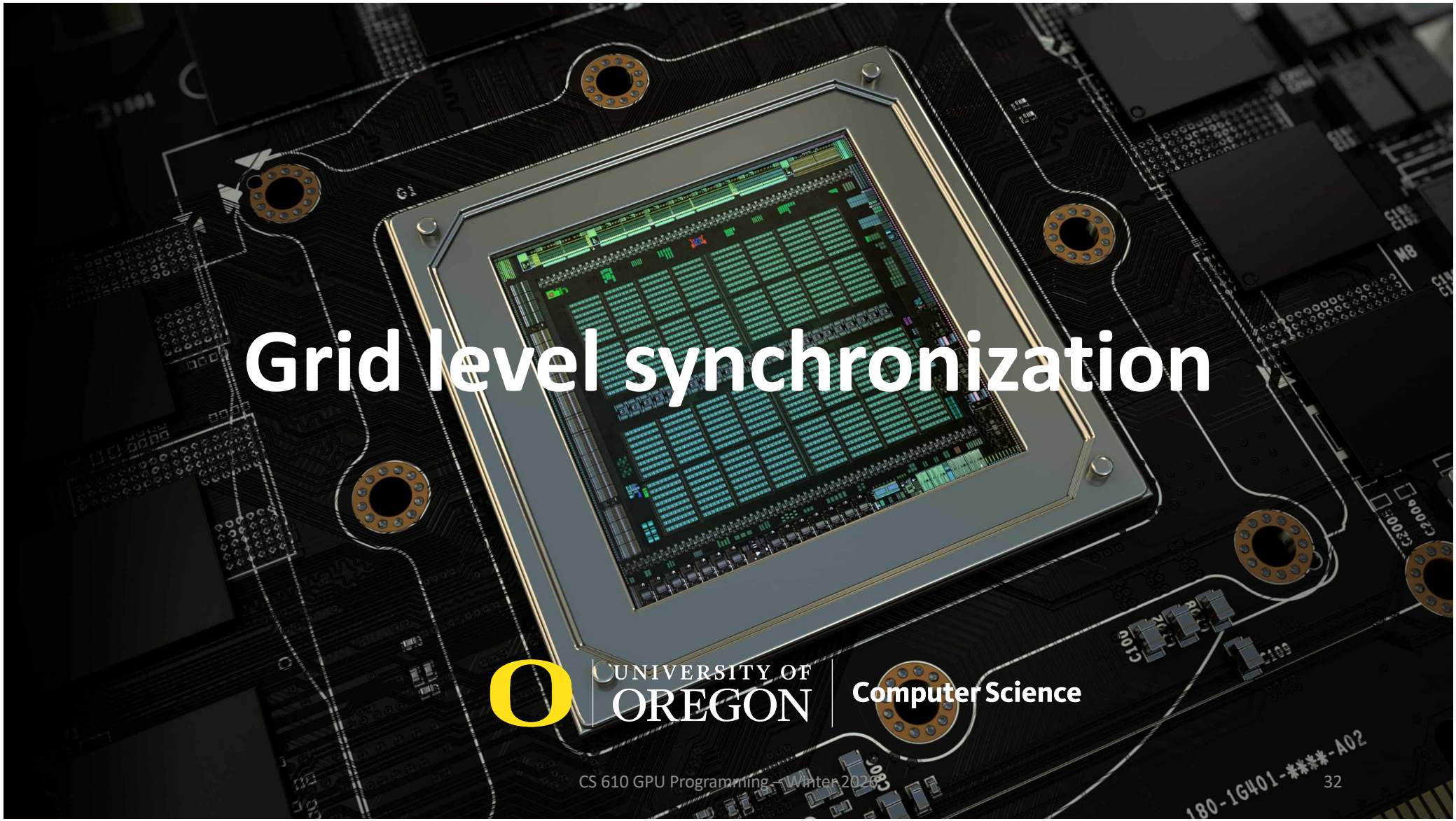
Which code can correctly run?

```
If (threadIdx.x < 16) {
    ……
    __syncthreads()
} else {
    …….
    __syncthreads()
}
```

```
If (blockIdx.x < 16) {
    ……
    __syncthreads()
} else {
    …….
    __syncthreads()
}
```

```
If (blockIdx.x < 16) {
    ……
} else {
    …….
}
 __syncthreads()
```

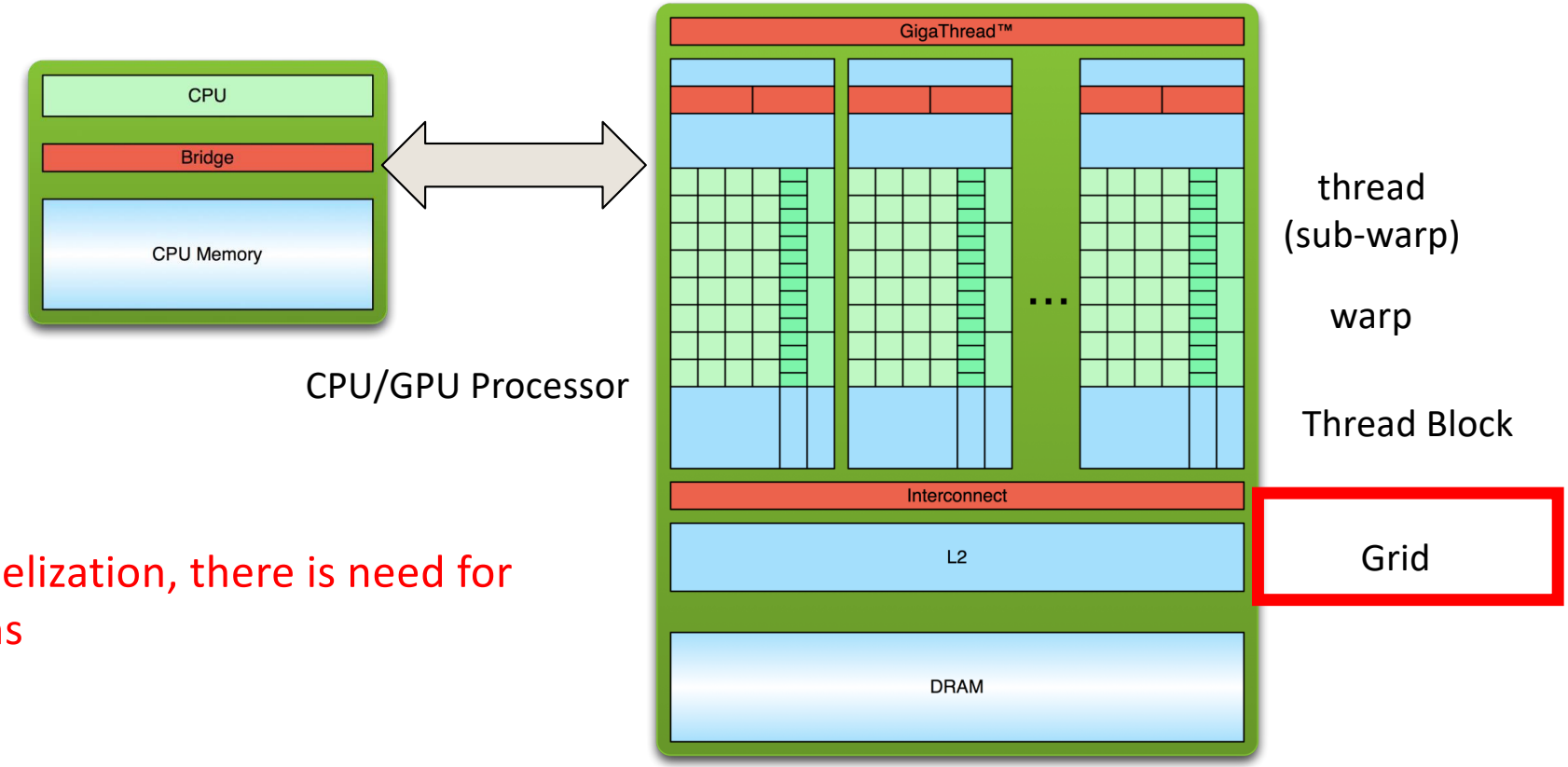
```
If (blockIdx.x < 16) {
    ……
    __syncthreads()
} else {
    …….
}
```



Grid level synchronization

UNIVERSITY OF OREGON | Computer Science

Parallelisms

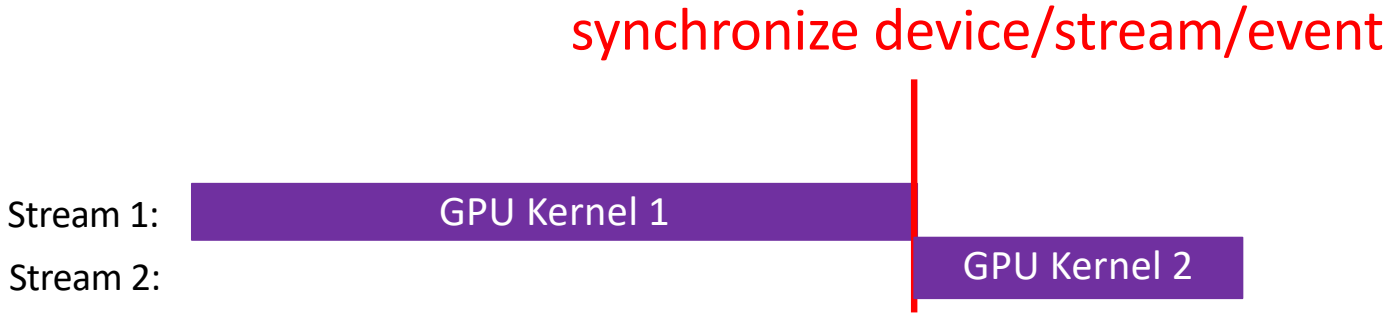


If there is parallelization, there is need for synchronizations

Grid level synchronization

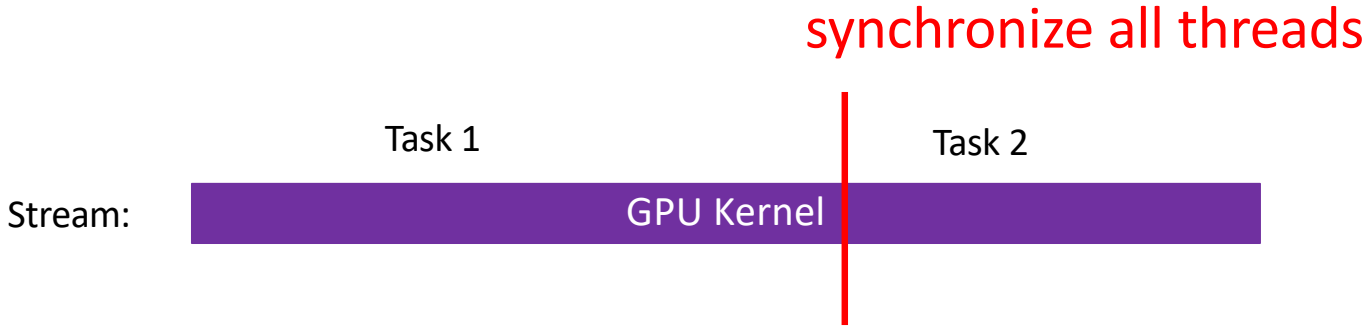
- Synchronize all threads in a grid
- Enforcing all instructions of all threads to synchronize
- Enable correct data sharing between threads in a grid

Method 1: sync outside kernel




```
Kernel1<<<16, 256, 0, stream1>>>(…parameters…);
cudaDeviceSynchronize();
Kernel2<<<16, 256, 0, stream2>>>(…parameters…);
```

Method 2: sync inside kernel




```
Kernel<<<16, 256, 0, stream>>>(…parameters…);
```

How to synchronize all thread inside kernel?


Cooperative Group
UNIVERSITY OF
OREGON
Computer Science

Cooperative Group

- A flexible model for synchronization and communication within groups of threads.
- Allows developers to express the granularity at which threads are communicating
- Achieving more efficient parallel algorithms

GRID GROUP

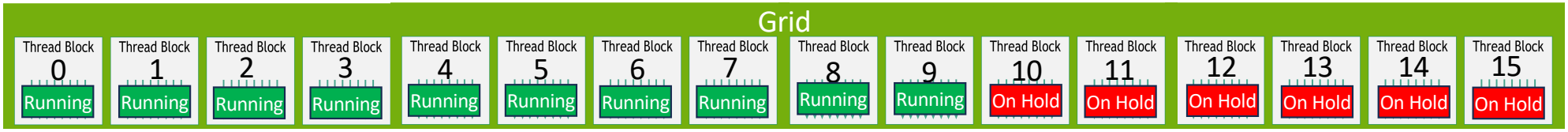
```
__global__kernel() {
    grid_group grid = this_grid();
    grid.sync();
    // all threads in a grid are now synced
}
```

Simplified way to sync

```
__global__kernel() {
    this_grid().sync();
    // all threads in a grid are now synced
}
```

Special condition must be met to enable grid sync: All thread blocks must be active

Review: TB scheduling strategy

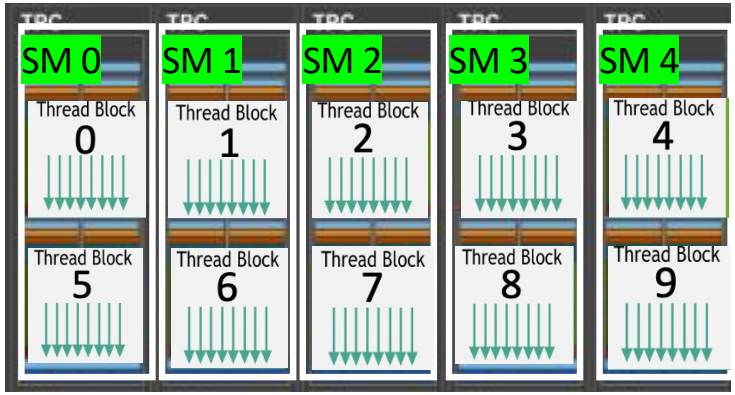


- Assume each SM can take *K* TBs and we have *N* SMs
- So, the GPU can execute *K*N* TBs at the same time

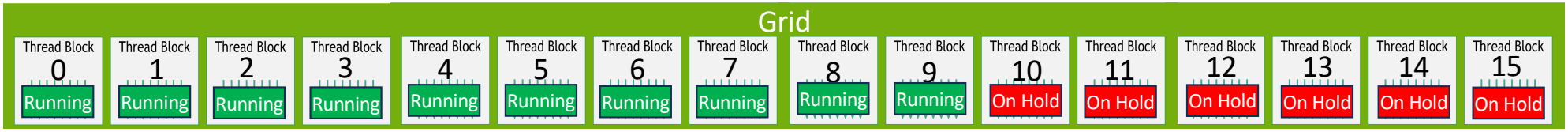
What if a grid contains more than *K*N* TBs?

At most *K*N* TBs will be running and rest TBs will be put on hold until some active TBs finished execution and release resources on SM. Then, TBs on hold will be assigned to SMs for execution

Time = t0



What happens if we call `grid.sync();`?



Remember:

- there is no context switch at TB level
- All works of a TB need to be finished before the SM releases the TB

What happens if we call `grid.sync();`?

Deadlock will occur

- Running TBs will be hold at grid.sync() forever (never released from SM)
- On-hold TBs will never get to execute, so they will never all grid.sync()

How to ensure all TBs are running?

Step 1:

Let CUDA calculate how many TBs can fit in a SM (i.e. K)

```
cudaOccupancyMaxActiveBlocksPerMultiprocessor (
        int* numBlocks,
        const void* func,
        int  blockSize,
        size_t dynamicSMemSize )
```

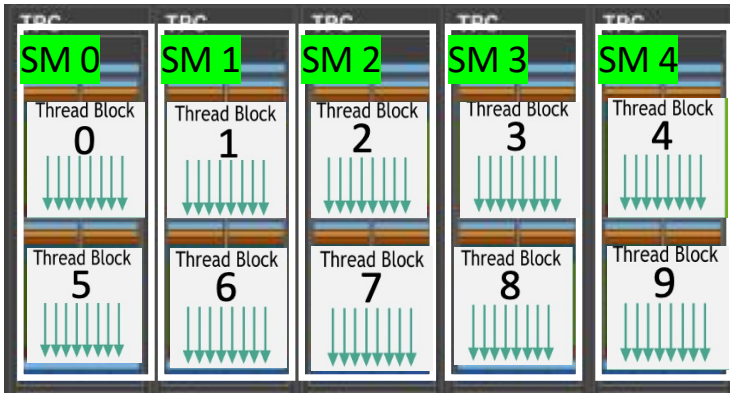
Parameters

`numBlocks`
- Returned occupancy

`func`
- Kernel function for which occupancy is calculated

`blockSize`
- Block size the kernel is intended to be launched with

`dynamicSMemSize`
- Per-block dynamic shared memory usage intended, in bytes



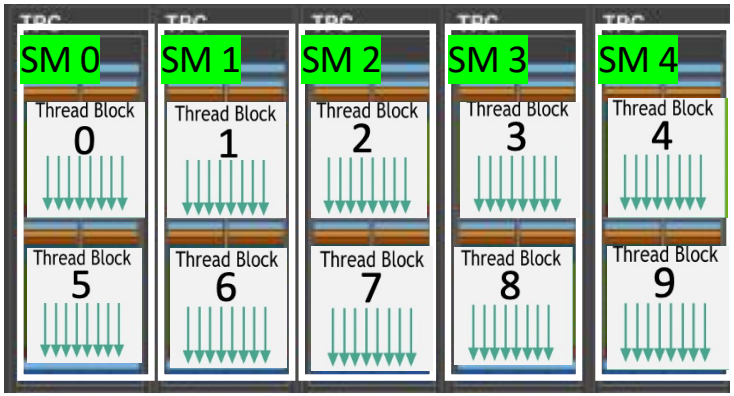
Assume each SM can take *K* TBs and we have *N* SMs
So, the GPU can execute *K*N* TBs at the same time

How to ensure all TBs are running?

Step 2:

Query the total number of SMs in the GPU

```
int dev = 0;
cudaDeviceProp deviceProp;
cudaGetDeviceProperties(&deviceProp, dev);
int numSM = deviceProp.multiProcessorCount
```



Assume each SM can take *K* TBs and we have *N* SMs
So, the GPU can execute *K*N* TBs at the same time

How to ensure all TBs are running?

Step 3:

Launch kernel with cooperative group enabled

```
cudaLaunchCooperativeKernel (
        const void* func,
        dim3 gridDim,
        dim3 blockDim,
        void** args,
        size_t sharedMem,
        cudaStream_t stream )
```

Parameters

`func`
- Device function symbol

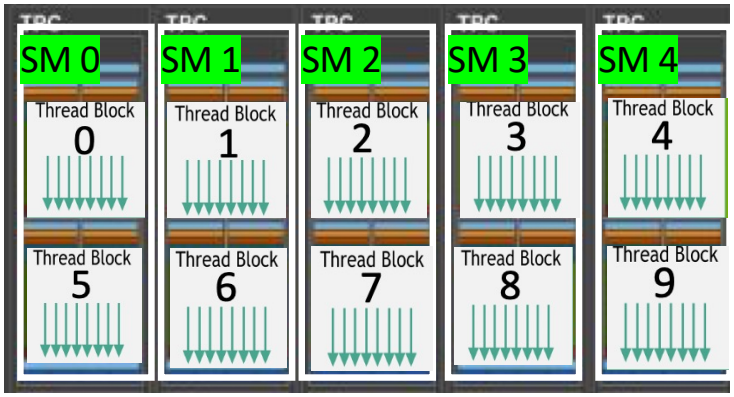
`gridDim`
- Grid dimentions

`blockDim`
- Block dimentions

`args`
- Arguments

`sharedMem`
- Shared memory

`stream`
- Stream identifier



Assume each SM can take *K* TBs and we have *N* SMs
So, the GPU can execute *K*N* TBs at the same time

Example

```
#include <cooperative_groups.h>
__global__ kernel(int * data) {
        // all threads work on data
        grid_group grid = this_grid();
        grid.sync(); // all threads in a grid are now synced
        // all thread work on data again
}
int numBlocksPerSm = 0;
int numThreads = 128;
cudaDeviceProp deviceProp;
cudaGetDeviceProperties(&deviceProp, dev);
cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocksPerSm, my_kernel, numThreads, 0);
int * data;
cudaMalloc((void**)&data, 100*sizeof(int));
void *kernelArgs[] = { (void*)&data };
dim3 dimBlock(numThreads, 1, 1);
dim3 dimGrid(deviceProp.multiProcessorCount*numBlocksPerSm, 1, 1);
cudaLaunchCooperativeKernel((void*)my_kernel, dimGrid, dimBlock, kernelArgs);
```

include header

Define kernel

Get device property

Get max # of TB/SM

Assign parameter

Calc. total # of TBs

Launch kernel

When using cooperative group

```
cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocksPerSm, my_kernel, numThreads, 0);
int * data;
cudaMalloc((void**)&data, 100*sizeof(int));
void *kernelArgs[] = { (void*)&data };
dim3 dimBlock(numThreads, 1, 1);
dim3 dimGrid(deviceProp.multiProcessorCount*numBlocksPerSm, 1, 1);
cudaLaunchCooperativeKernel((void*)my_kernel, dimGrid, dimBlock, kernelArgs);
```

Get max # of TB/SM

Assign parameter

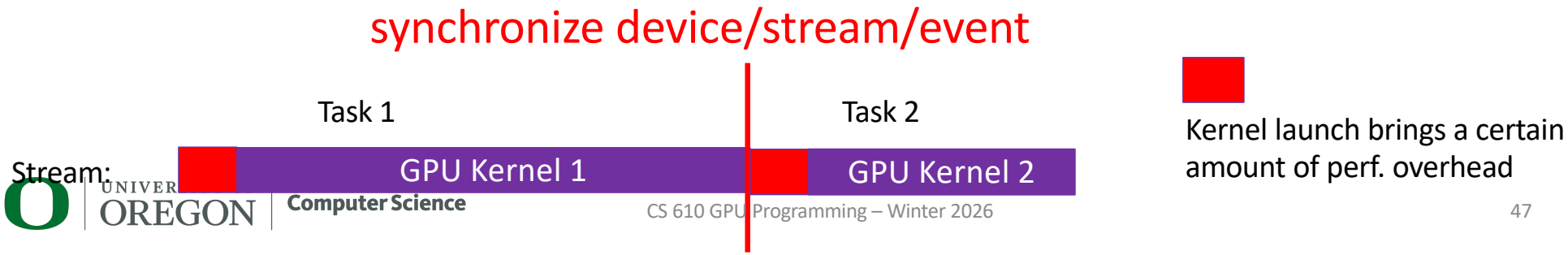
Calc. total # of TBs

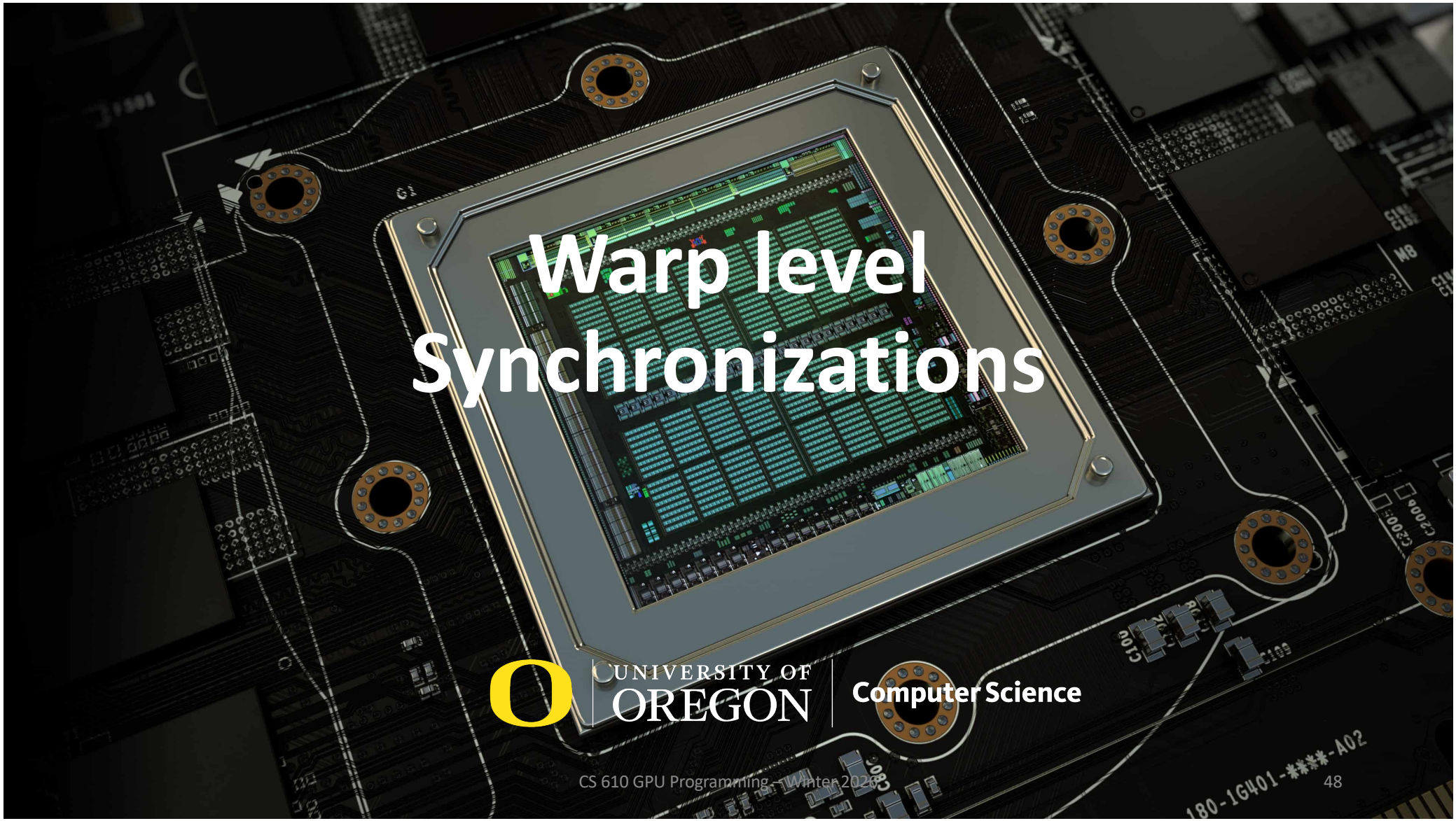
Launch kernel

- The total number of TBs is no longer determined by the developers
- Your algorithm need to accommodate to this restriction
  - Add loops for larger data
  - Making runtime decision on whether to use cooperative group or rely on out-of-kernel sync
- Limit the usage of shared memory and registers to increase the # of active TBs per SM

Out-of-kernel sync vs. in-kernel grid sync

	Out-of-kernel sync (device/stream/event sync)	In-kernel sync (cooperative group grid.sync)
Advantage	• No restriction on total number of TBs • Algorithms can be easily partitioned into several kernels	• Faster: No need to have multiple kernel launch, so no extra kernel launch overhead.
Disadvantage	• Slower: Algorithm needs multiple kernel launches. Each kernel launch bring extra kernel launch overhead.	• Restriction on the total number of TBs (restriction on total parallelism) • Algorithms can be hard to be put in one kernel

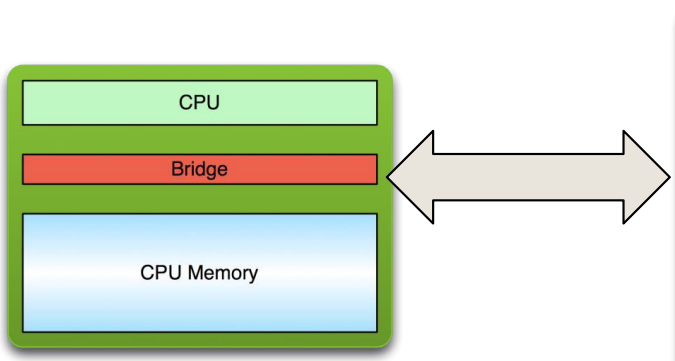




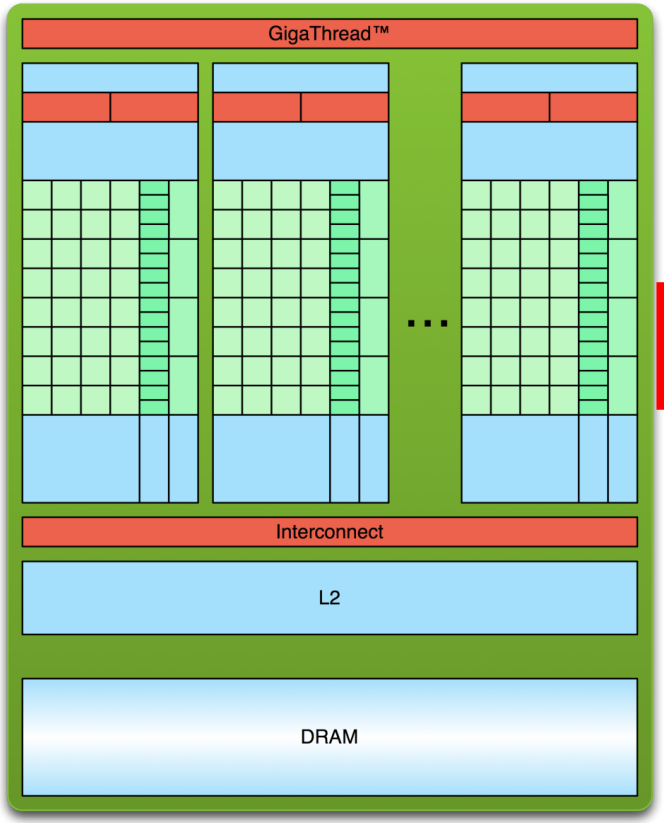
Warp level Synchronizations

UNIVERSITY OF OREGON | Computer Science

Parallelisms



CPU/GPU Processor



thread
(sub-warp)

warp

Thread Block

Grid

If there is parallelization, there is need for synchronizations

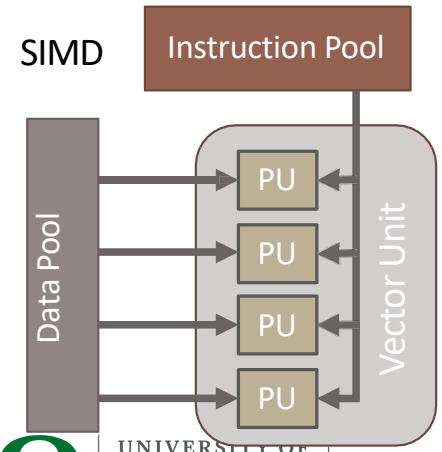
Thread block level synchronizations

- Synchronize all thread in a warp
- Enforcing all instructions of all threads before synchronization are finished
- Enable correct data sharing between threads in warp

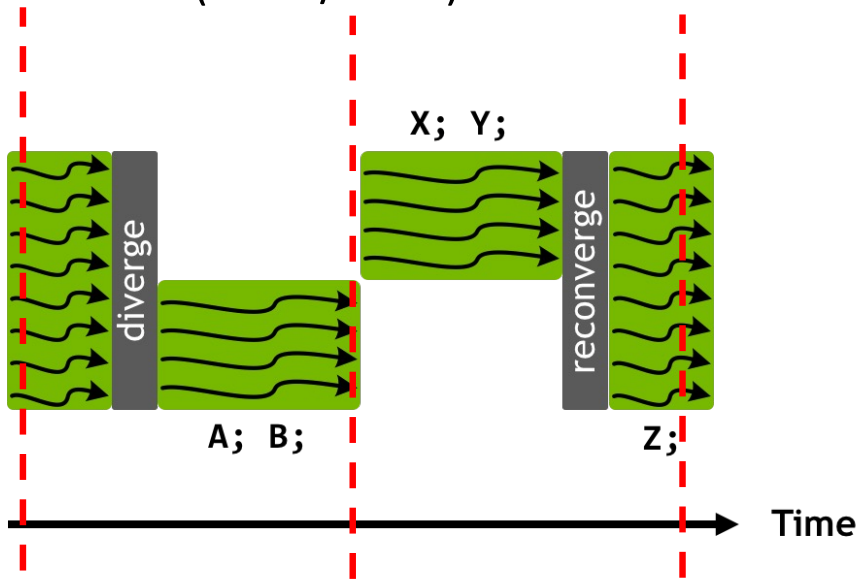
Why do we need to sync a warp?

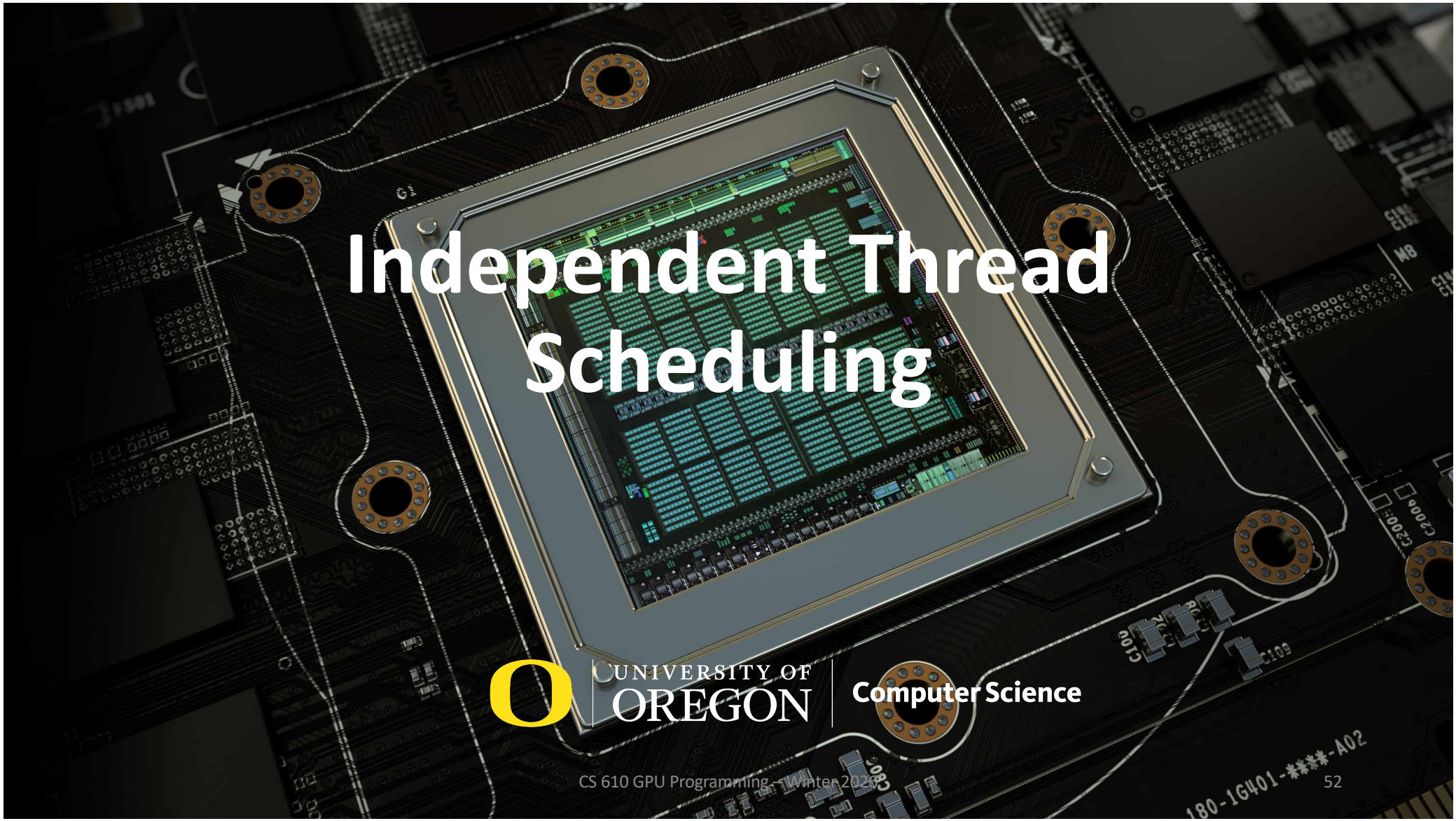
Remember we said

- A warp is the basic unit for scheduling
- A warp has a single program counter
- Threads a warp always execute the same instruction (SIMD/SIMT)
- Threads a warp is always in sync




```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



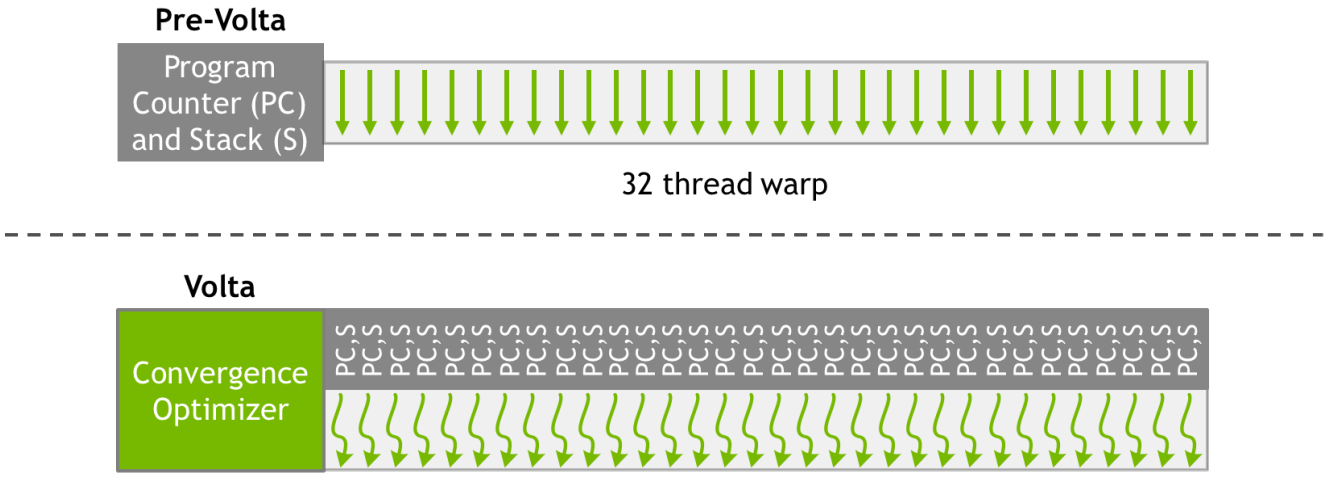


Independent Thread Scheduling

UNIVERSITY OF OREGON | Computer Science

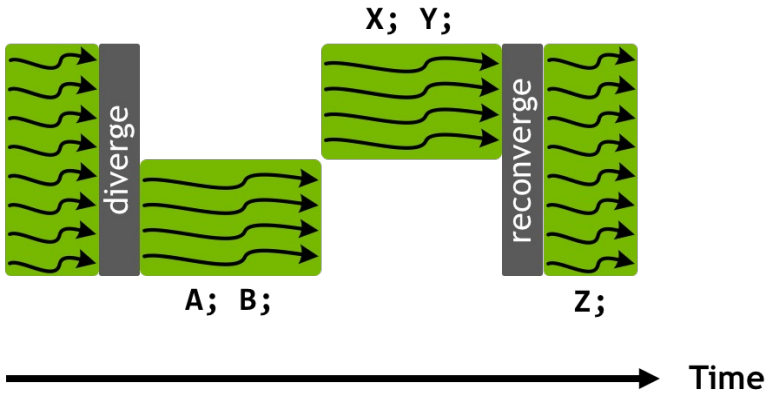
Independent Thread Scheduling (ITS)

- Starting from Volta GPU (e.g. Volta, Ampere, Turing, Hooper,…), each thread in a warp has their own program counter
- Each thread can make progress on their own without waiting for others



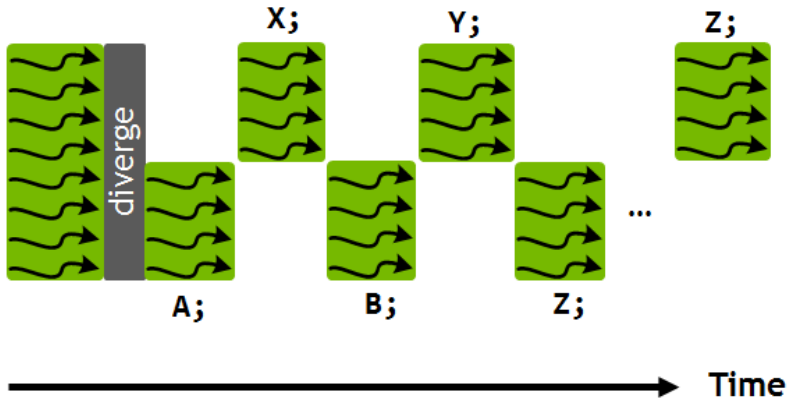
Pre-volta

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



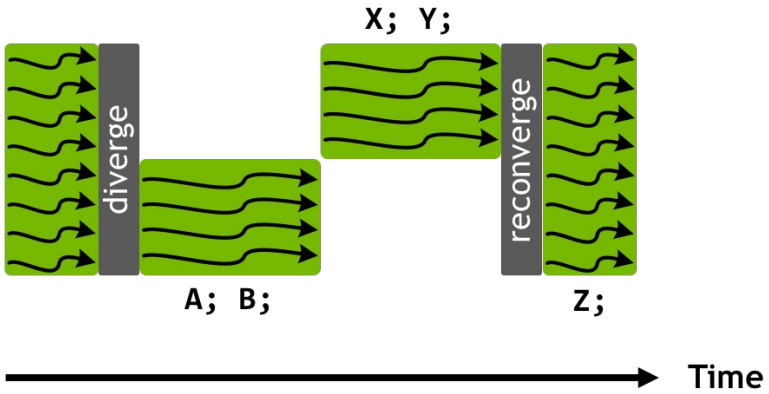
Volta and later

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



Pre-volta

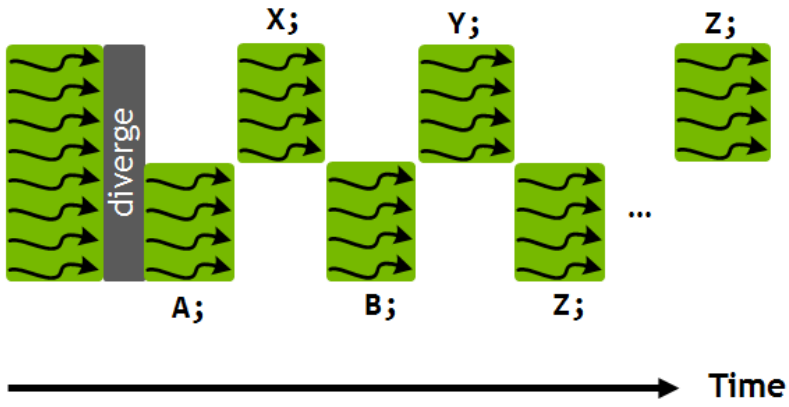
```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



- Tracking thread state in aggregate for the whole warp
- This loss of concurrency means that threads from the same warp in divergent regions
- Prohibit fine-grained sharing of data
- E.g., impossible for thread 0 to shared data with thread 5 in the divergent regions

Volta and later

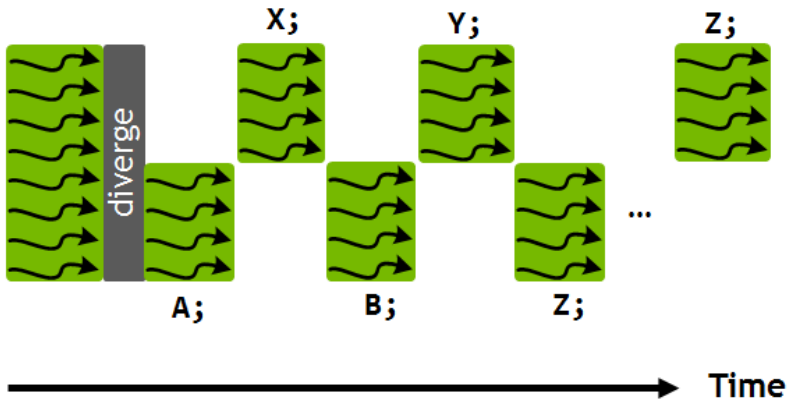
```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



- Independent thread scheduling allows the GPU to yield execution of any thread
- A schedule optimizer which determines how to group active threads from the same warp together into SIMT units.
- Threads can now diverge and reconverge at sub-warp granularity
- GPU will still group together threads which are executing the same code and run them in parallel

Volta and later

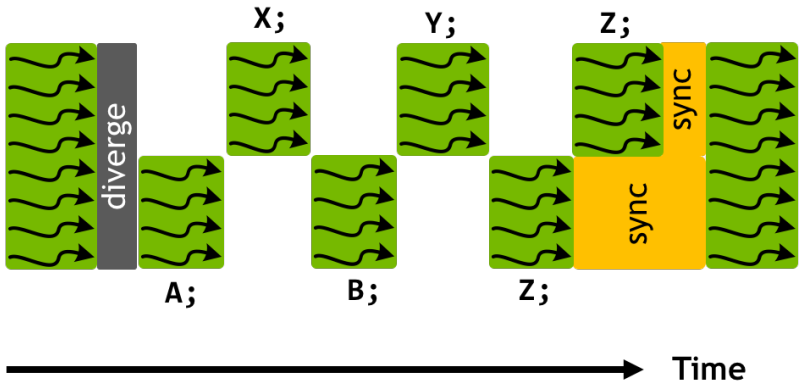
```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



- For example, threads may not converge/sync when executing Z
- To force synchronization, call `__syncwarp()`

Volta and later

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
__syncwarp()
```



- For example, threads may not converge/sync when executing Z
- To force synchronization, call `__syncwarp()`

Will these work with Independent Thread Scheduling?

- If (threadIdx.x % 2 == 0) {
- ……
- __syncthreads();
- } else {
- …….
- __syncthreads();
- }

- If (threadIdx.x % 2 == 0) {
- ……
- __syncwarp();
- } else {
- …….
- __syncwarp();
- }

THREAD BLOCK TILE

A subset of threads of a thread block, divided into tiles in row-major order

```
thread_block_tile<32> tile32 = tiled_partition<32>(this_thread_block());
```

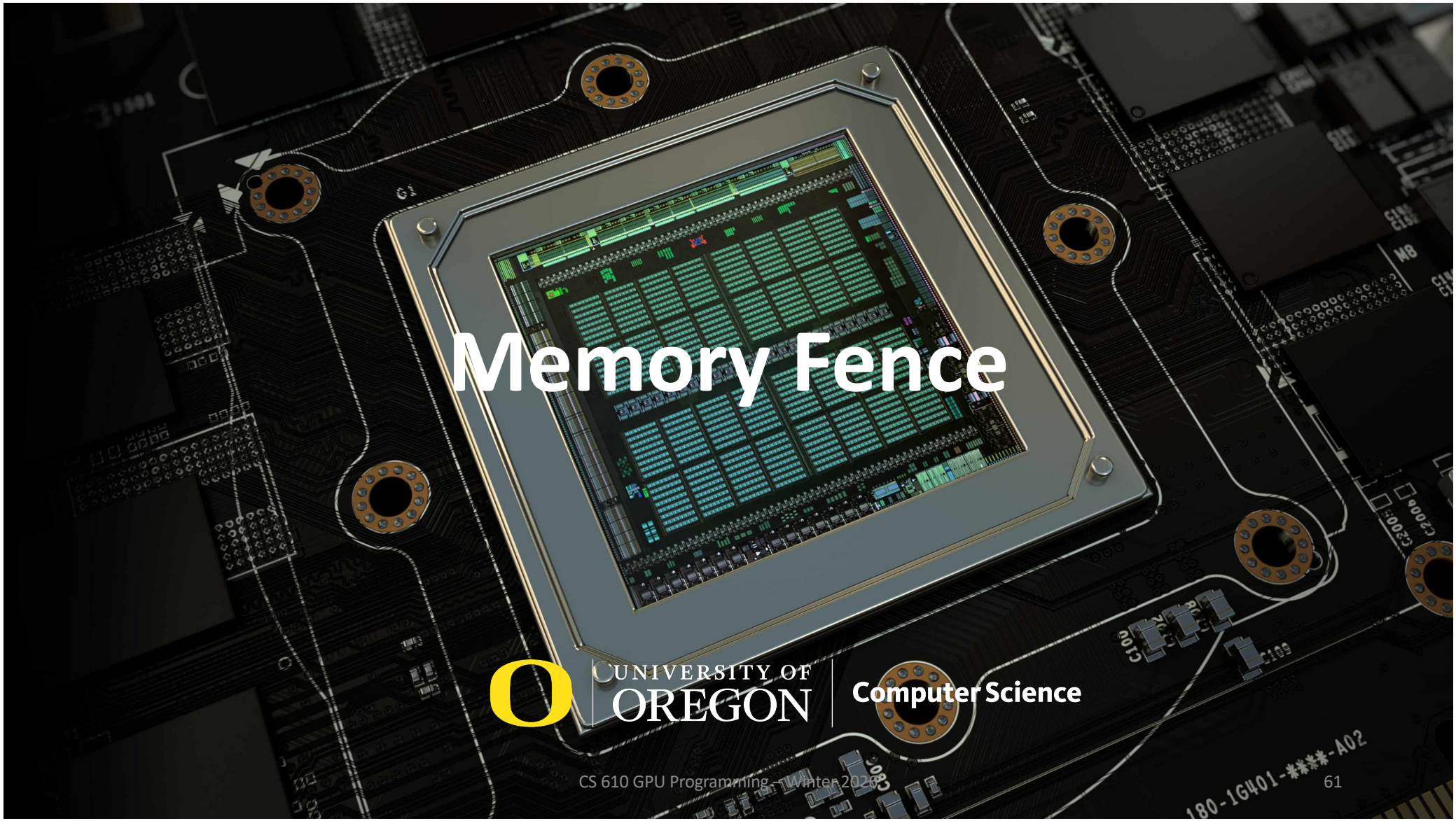
```
thread_block_tile<4> tile4 = tiled_partition<4>(this_thread_block());
```

Exposes additional functionality:

```
.shfl()
.shfl_down()
.shfl_up()
.shfl_xor()
```

```
.any()
.all()
.ballot()
.match_any()
.match_all()
```

Size known at compile time = fast!


Memory Fence
UNIVERSITY OF
OREGON
Computer Science

Weakly-ordered memory model

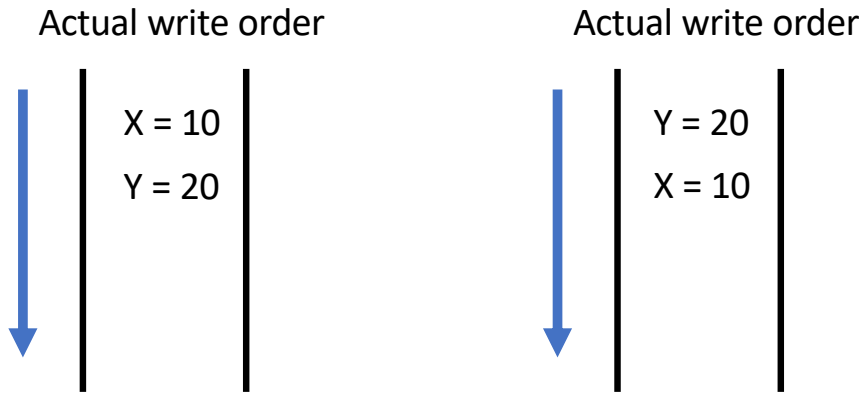
- The order in which a CUDA thread writes data to memory is not guaranteed
- This applies to
  - shared memory
  - global memory
  - pinned host memory
  - memory of a peer device (other GPU)

Weakly-ordered memory model

- The order in which a CUDA thread writes data to memory is not guaranteed

A thread executes:

```
__device__ int X = 1, Y = 2;
__device__ void writeXY()
{
   X = 10;
   Y = 20;
}
```



Since the writes to X and Y have no dependency, they can be executed in any order

Weakly-ordered memory model

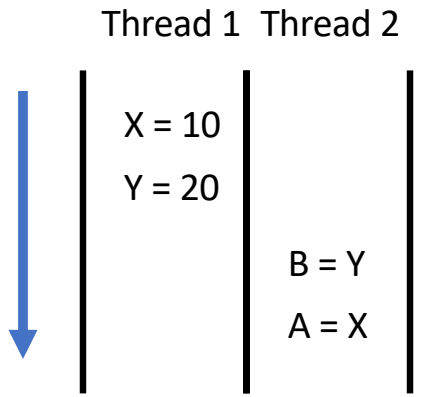
```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   int A = X;
}
```

What would be the result value of A and B?

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   int A = X;
}
```

What would be the result value of A and B?

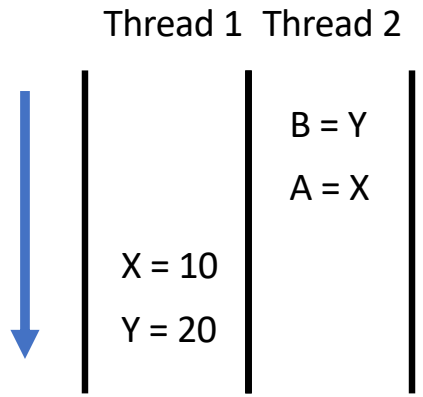


B = 20
A = 10

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   int A = X;
}
```

What would be the result value of A and B?



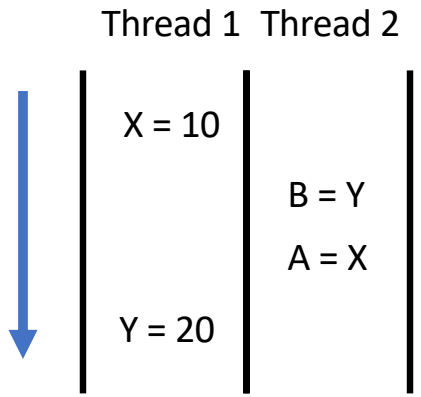
B = 2

A = 1

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   int A = X;
}
```

What would be the result value of A and B?

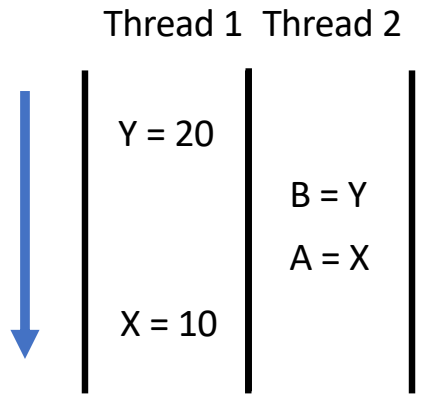


B = 2
A = 10

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   int A = X;
}
```

What would be the result value of A and B?



B = 20

A = 1

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   int A = X;
}
```

What would be the result value of A and B?

B = 20, A = 10
B = 2, A = 1
B = 20, A = 1
B = 2, A = 10

All possible output

Memory fence operation

- Memory fence functions can be used to enforce a sequentially-consistent ordering on memory accesses.
- Memory fence functions differ in the scope in which the orderings are enforced
  - Thread Block
  - Device
  - System

Memory fence (scope: thread block)

```
void __threadfence_block();
```

- All writes made by the calling thread before __threadfence_block() are observed by all threads in the thread block as occurring before all writes made by the calling thread after __threadfence_block();
- All reads made by the calling thread before __threadfence_block() are ordered before all reads after __threadfence_block().

Memory fence (scope: device)

```
void __threadfence();
```

- All writes made by the calling thread before __threadfence() are observed by all threads in the device as occurring before all writes made by the calling thread after __threadfence();
- All reads made by the calling thread before __threadfence() are ordered before all reads after __threadfence().

Memory fence (scope: system)

```
void __threadfence_system();
```

- All writes made by the calling thread before __threadfence_system() are observed by all threads in the system as occurring before all writes made by the calling thread after __threadfence_system();
- All reads made by the calling thread before __threadfence_system() are ordered before all reads after __threadfence_system().

Weakly-ordered memory model

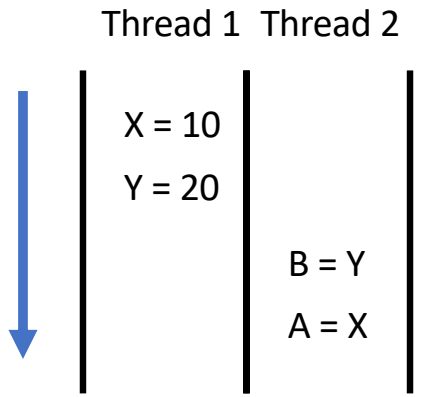
```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   __threadfence();
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   __threadfence();
   int A = X;
}
```

What would be the result value of A and B?

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   __threadfence();
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   __threadfence();
   int A = X;
}
```

What would be the result value of A and B?



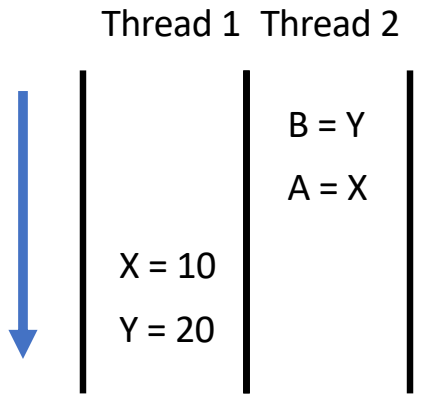
B = 20

A = 10

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   __threadfence();
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   __threadfence();
   int A = X;
}
```

What would be the result value of A and B?



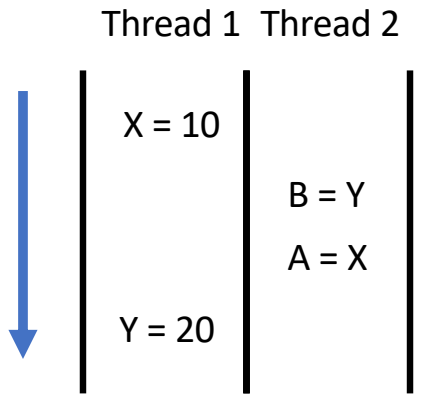
B = 2

A = 1

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   __threadfence();
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   __threadfence();
   int A = X;
}
```

What would be the result value of A and B?



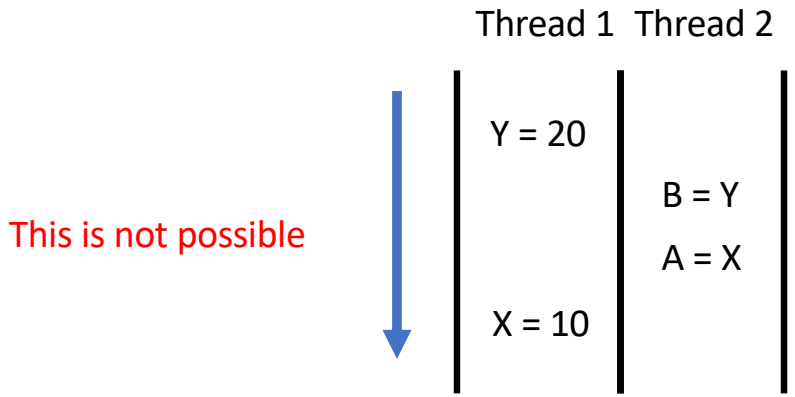
B = 2

A = 10

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   __threadfence();
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   __threadfence();
   int A = X;
}
```

What would be the result value of A and B?



B = 20

A = 1

Weakly-ordered memory model

```
__device__ int X = 1, Y = 2;

// Thread 1 executes:
__device__ void writeXY()
{
   X = 10;
   __threadfence();
   Y = 20;
}

// Thread 2 executes:
__device__ void readXY()
{
   int B = Y;
   __threadfence();
   int A = X;
}
```

What would be the result value of A and B?

B = 20, A = 10

B = 2, A = 1

~~B = 20, A = 1~~

B = 2, A = 10

All possible output

Example

```
__device__ int data;
__device__ int ready;

__global__ void kernel() {
   if (blockIdx.x == 0) {
      // Producer
      data = 42;
      ready = 1;   // signal
   } else {
      // Consumer
      if (ready == 1) {
         printf("%d\n", data);
      }
   }
}
```

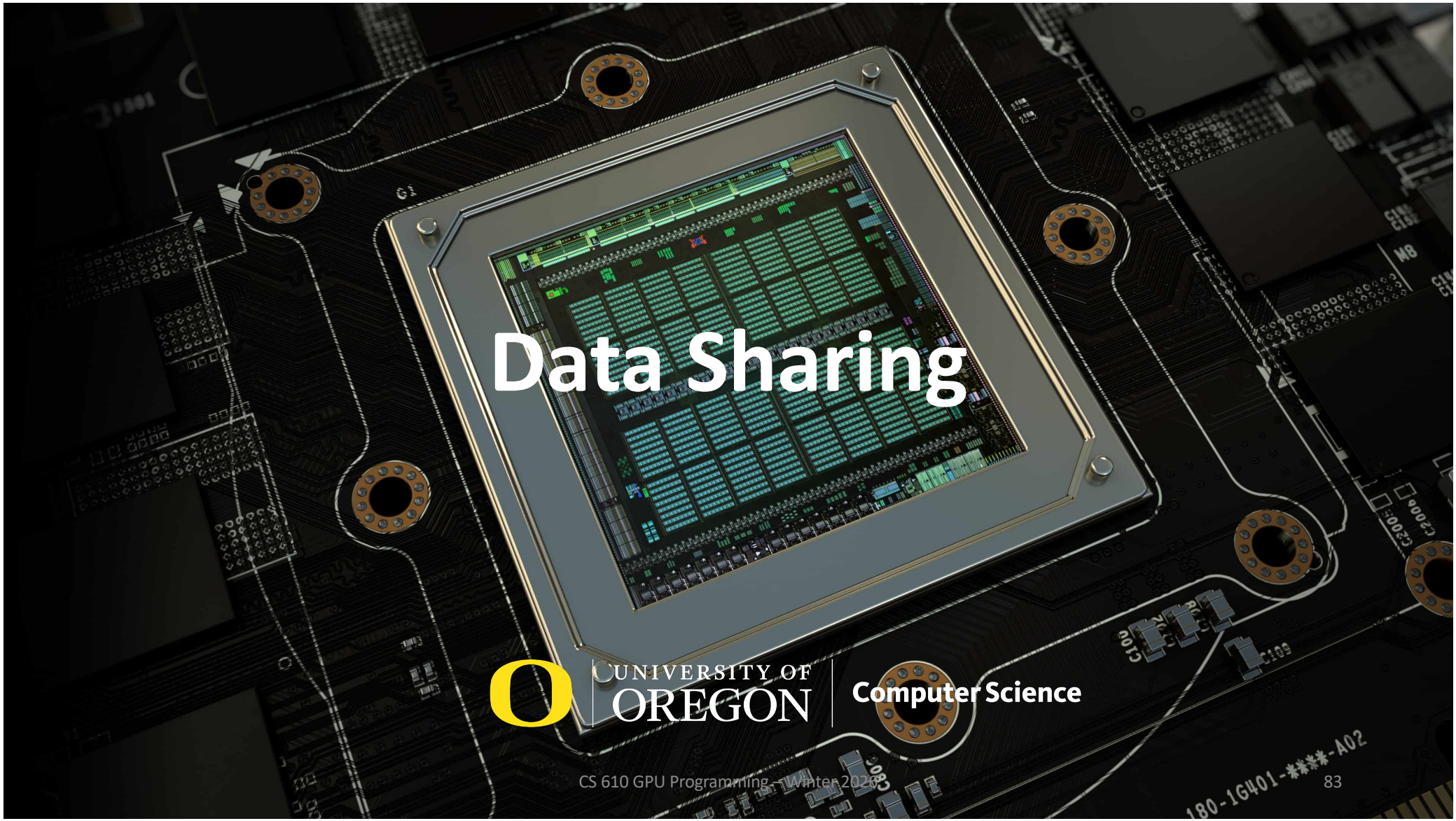
What will consumer print?

Example

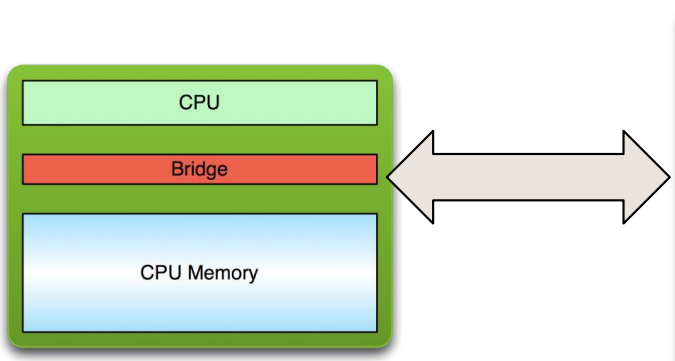
```
__device__ int data;
__device__ int ready;

__global__ void kernel() {
   if (blockIdx.x == 0) {
      // Producer
      data = 42;
      __threadfence();
      ready = 1;   // signal
   } else {
      // Consumer
      if (ready == 1) {
         printf("%d\n", data);
      }
   }
}
```

Memory fence vs. Sync.

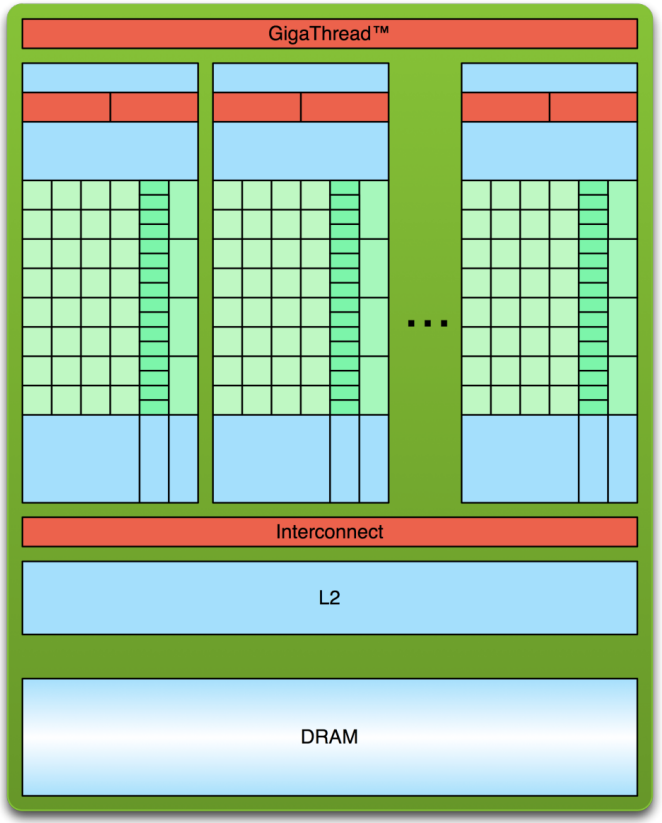
	Memory Fence	Synchronization
Advantage	• Faster	• Ensure executing order across multiple threads
Disadvantage	• Only ensure write/read ordering for each thread • Does not ensure executing order between threads	• Slower


Data Sharing
UNIVERSITY OF
OREGON
Computer Science

Parallelisms



CPU/GPU Processor



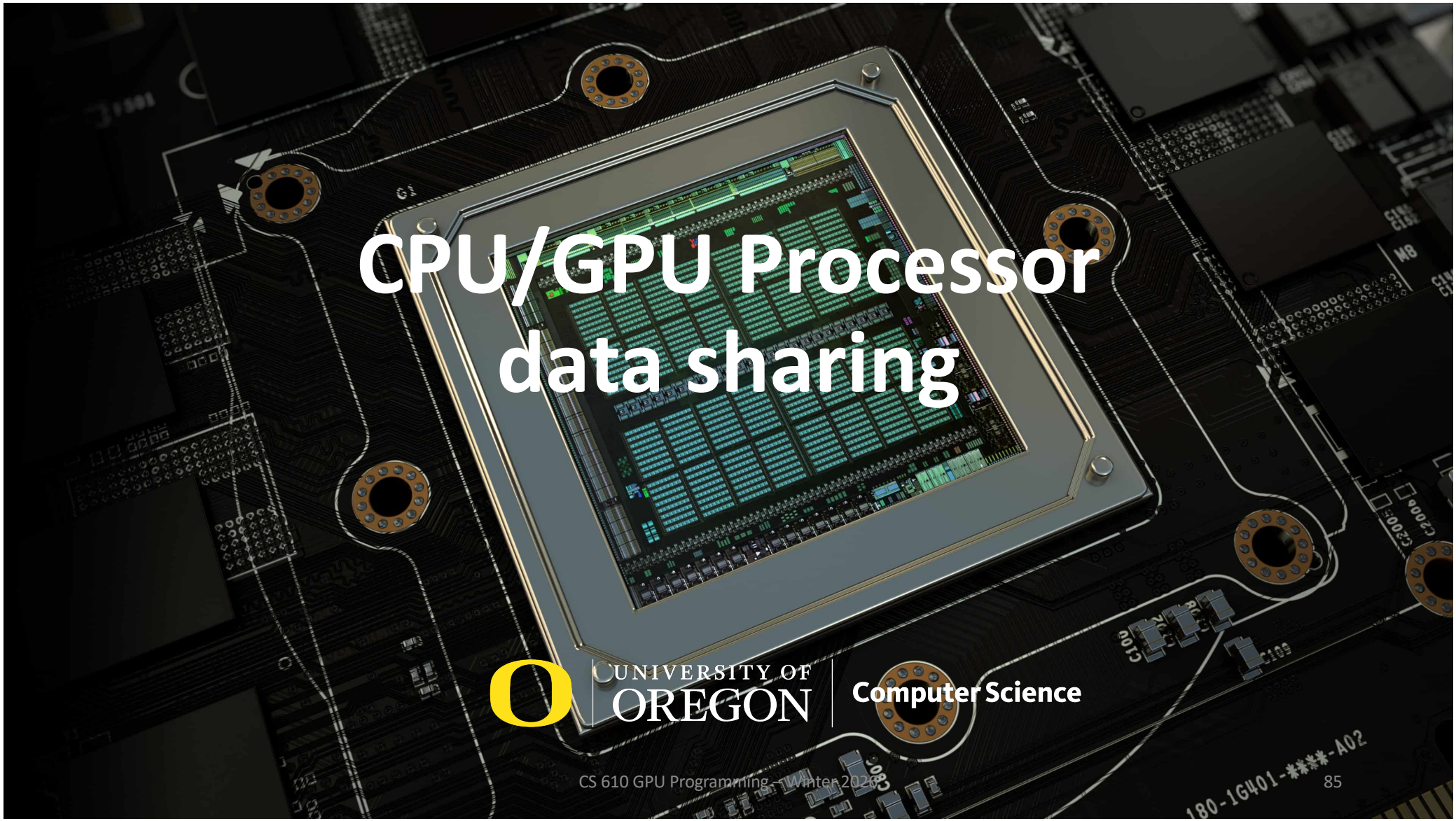
thread
(sub-warp)

warp

Thread Block

Grid

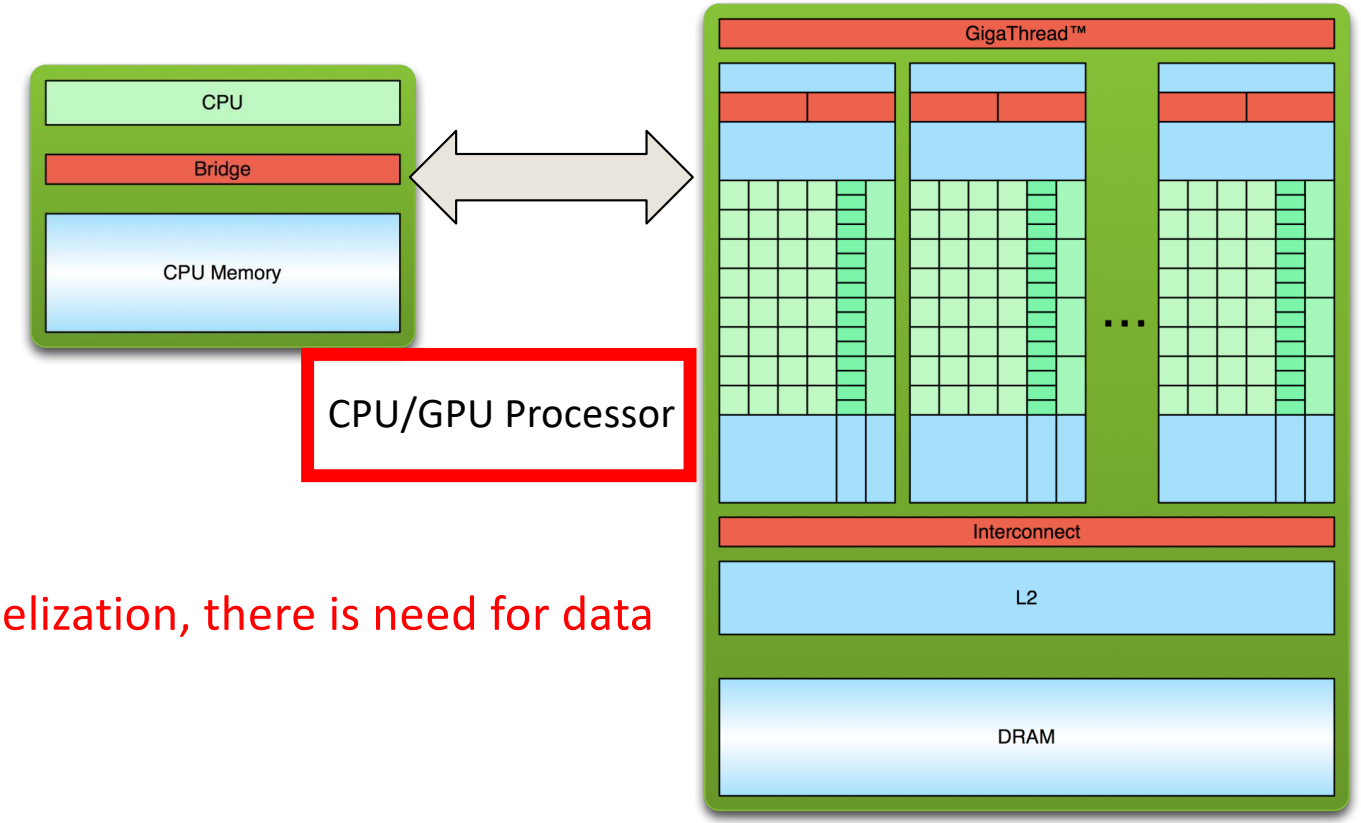
If there is parallelization, there is need for data sharing



CPU/GPU Processor data sharing

UNIVERSITY OF OREGON | Computer Science

Parallelisms



If there is parallelization, there is need for data sharing

CPU/GPU Processor data sharing

- Sharing data between CPU and GPU
- CPU and GPU have separate memory
- Explicit or implicit memory copy is necessary for sharing data

Memory Copying (review)

- Once memory has been allocated we need to copy data to it and from it.
- **`cudaMemcpy()`** transfers memory from the host to device to host and vice versa

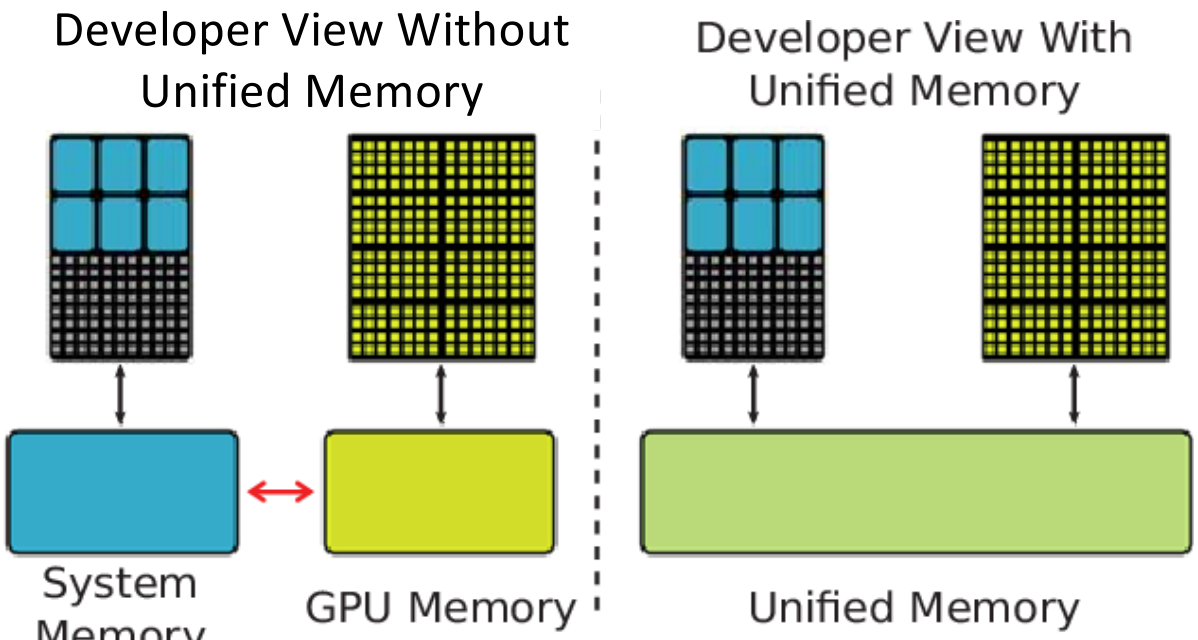
```
cudaMemcpy(array_device, array_host,
 N*sizeof(float), cudaMemcpyHostToDevice);
```

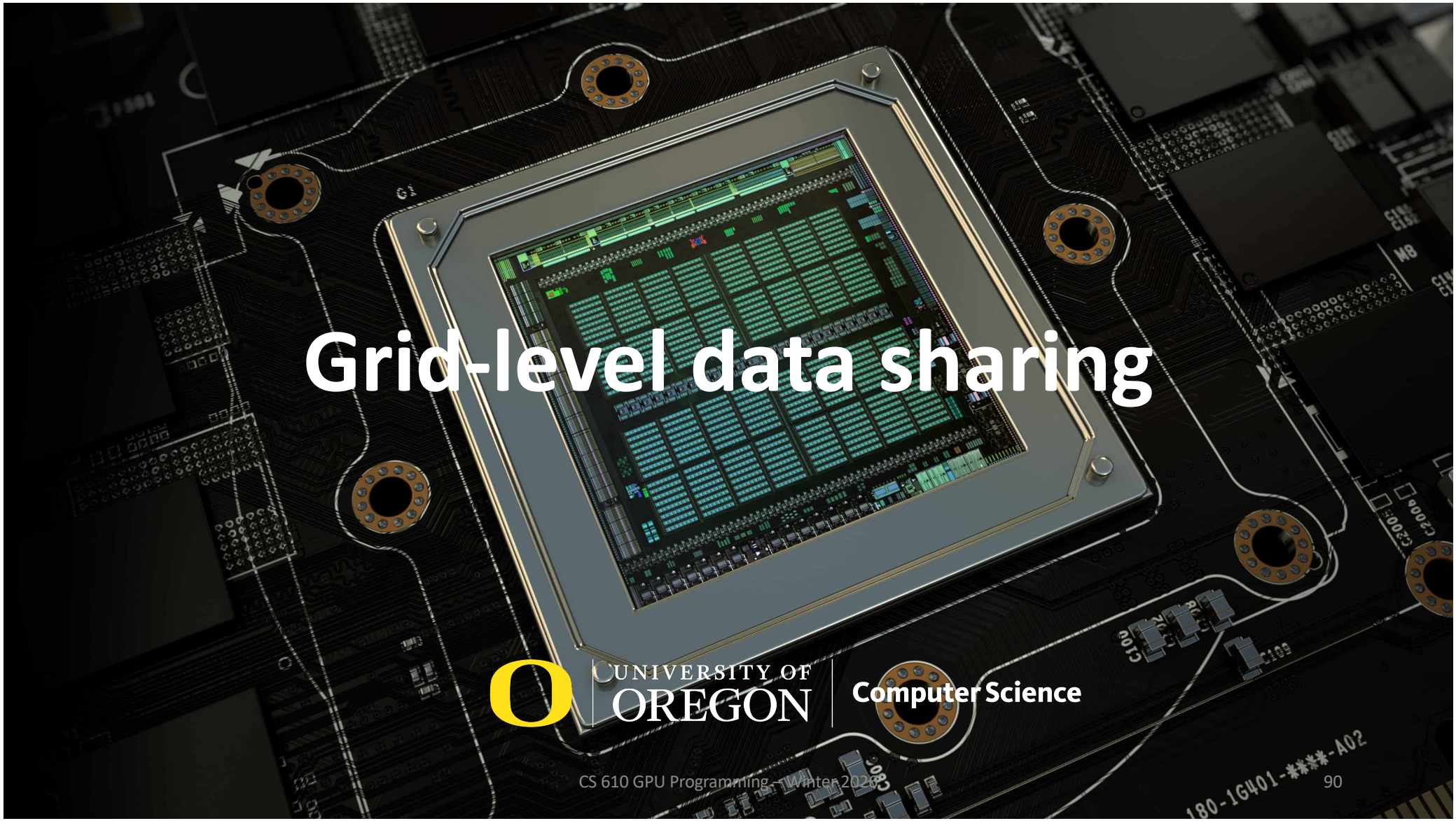
```
cudaMemcpy(array_host, array_device,
 N*sizeof(float), cudaMemcpyDeviceToHost);
```

- First argument is always the **destination** of transfer
- Transfers are relatively slow and should be minimised where possible

Unified Memory (review)

- Unified Memory = CUDA Managed Memory
- Accessible by both CPU and GPU

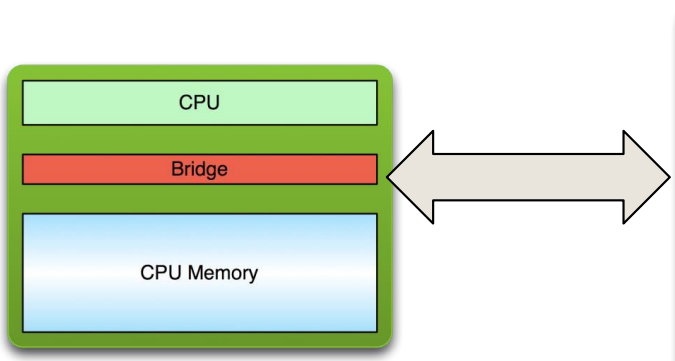




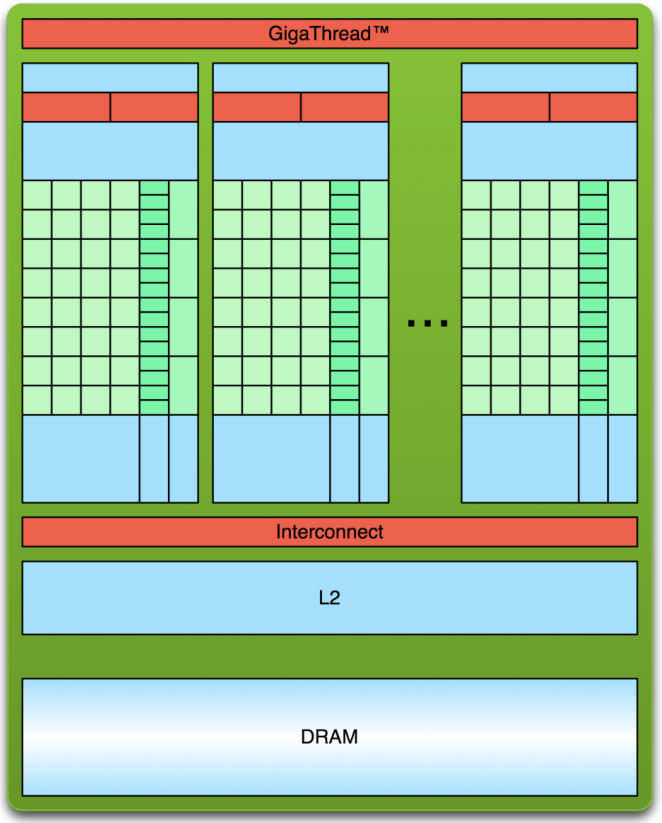
Grid-level data sharing

UNIVERSITY OF OREGON | Computer Science

Parallelisms



CPU/GPU Processor



thread
(sub-warp)

warp

Thread Block

Grid

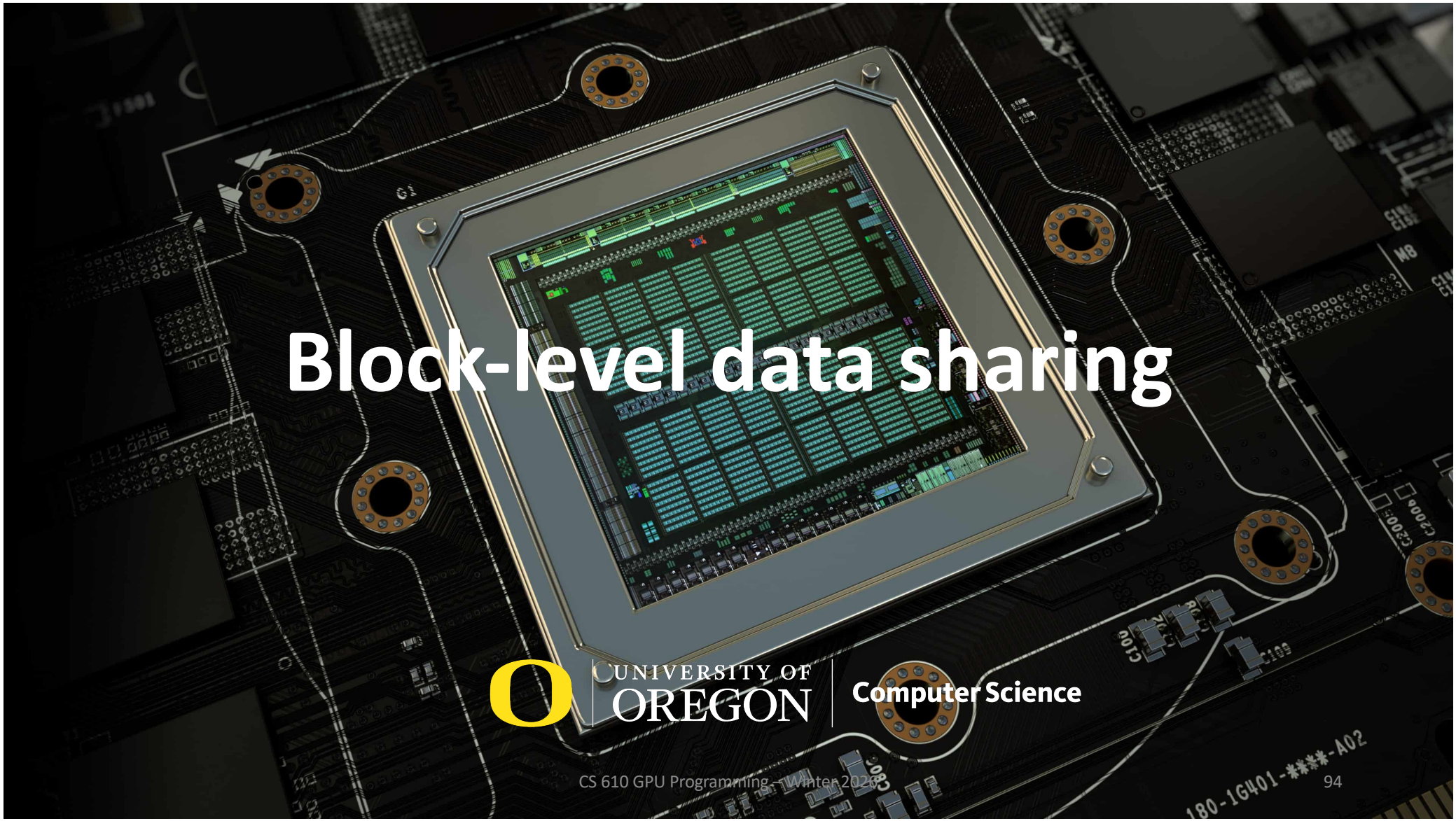
If there is parallelization, there is need for data sharing

Grid-level data sharing

- Sharing data between threads in a grid
- Threads may from different thread blocks
- Global memory is the only common accessible memory

Grid-level data sharing

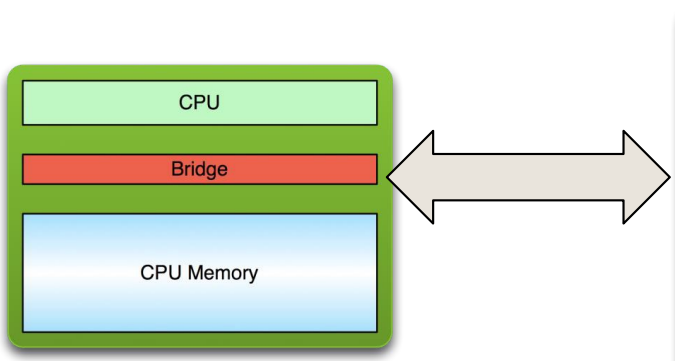
```
__global__kernel(int * data) {
    grid_group grid = this_grid();
    if (blockIdx.x == 0) {
      // write data to global memory
    }
    grid.sync();
    if (blockIdx.x == 1) {
      // read data from global memory
    }

}
```



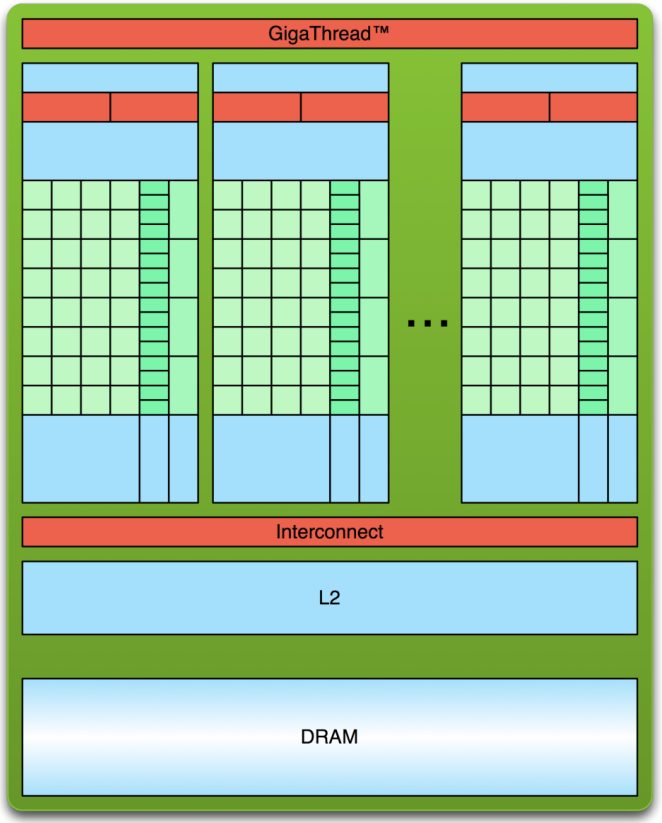
Block-level data sharing

UNIVERSITY OF OREGON | Computer Science

Parallelisms



CPU/GPU Processor



thread
(sub-warp)

warp

Thread Block

Grid

If there is parallelization, there is need for data sharing

Block-level data sharing

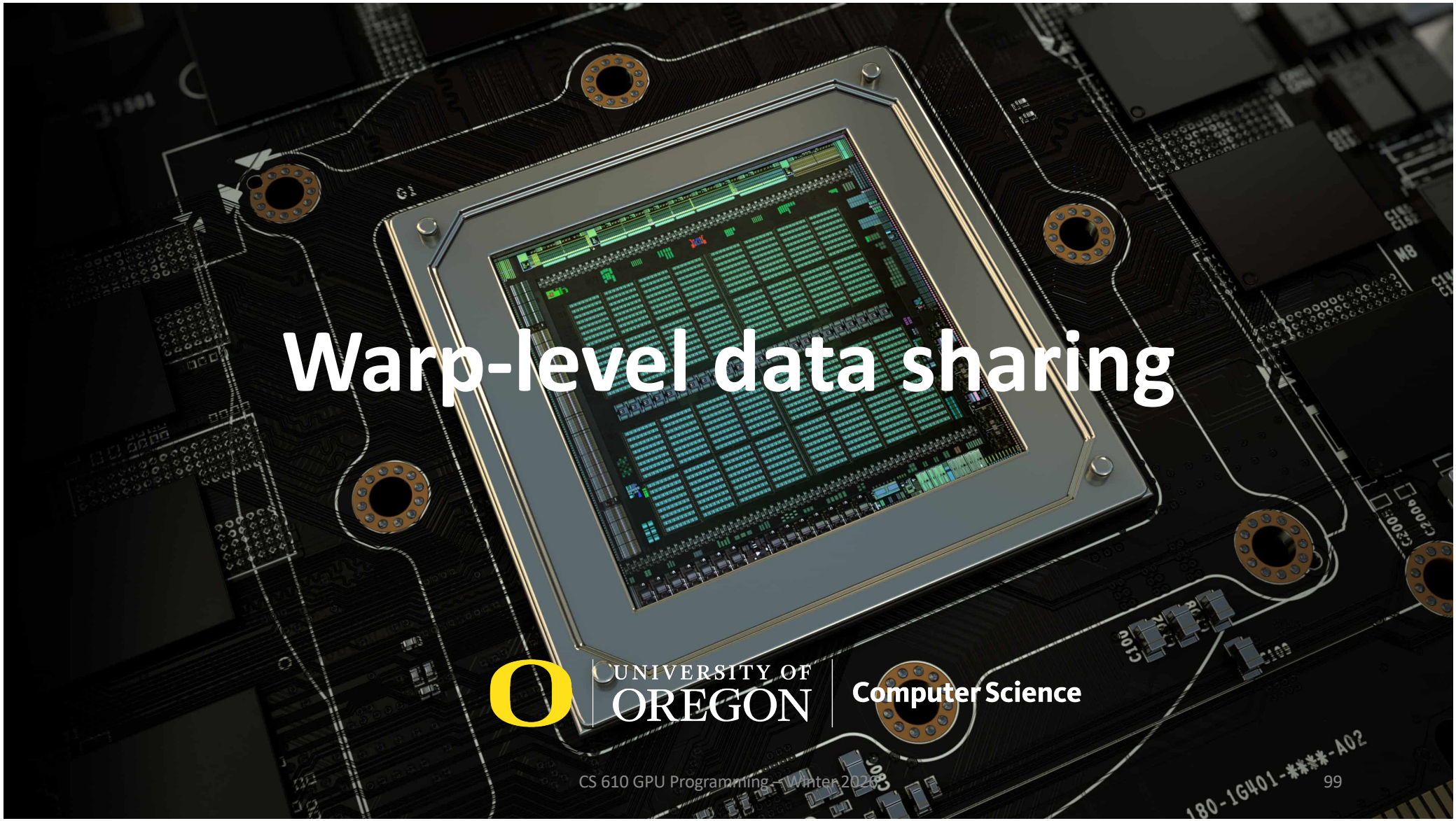
- Sharing data between threads that come from the same thread block
- Two common accessible memory
  - Shared memory: faster, less space (65K-225K)
  - Global memory: slower, larger space

Block-level data sharing

```
__global__kernel(int * data) {
    if (threadIdx.x == 0) {
       // write data to global memory
    }
    __syncthreads();
    if (threadIdx.x == 1) {
       // read data from global memory
    }

}
```

Block-level data sharing

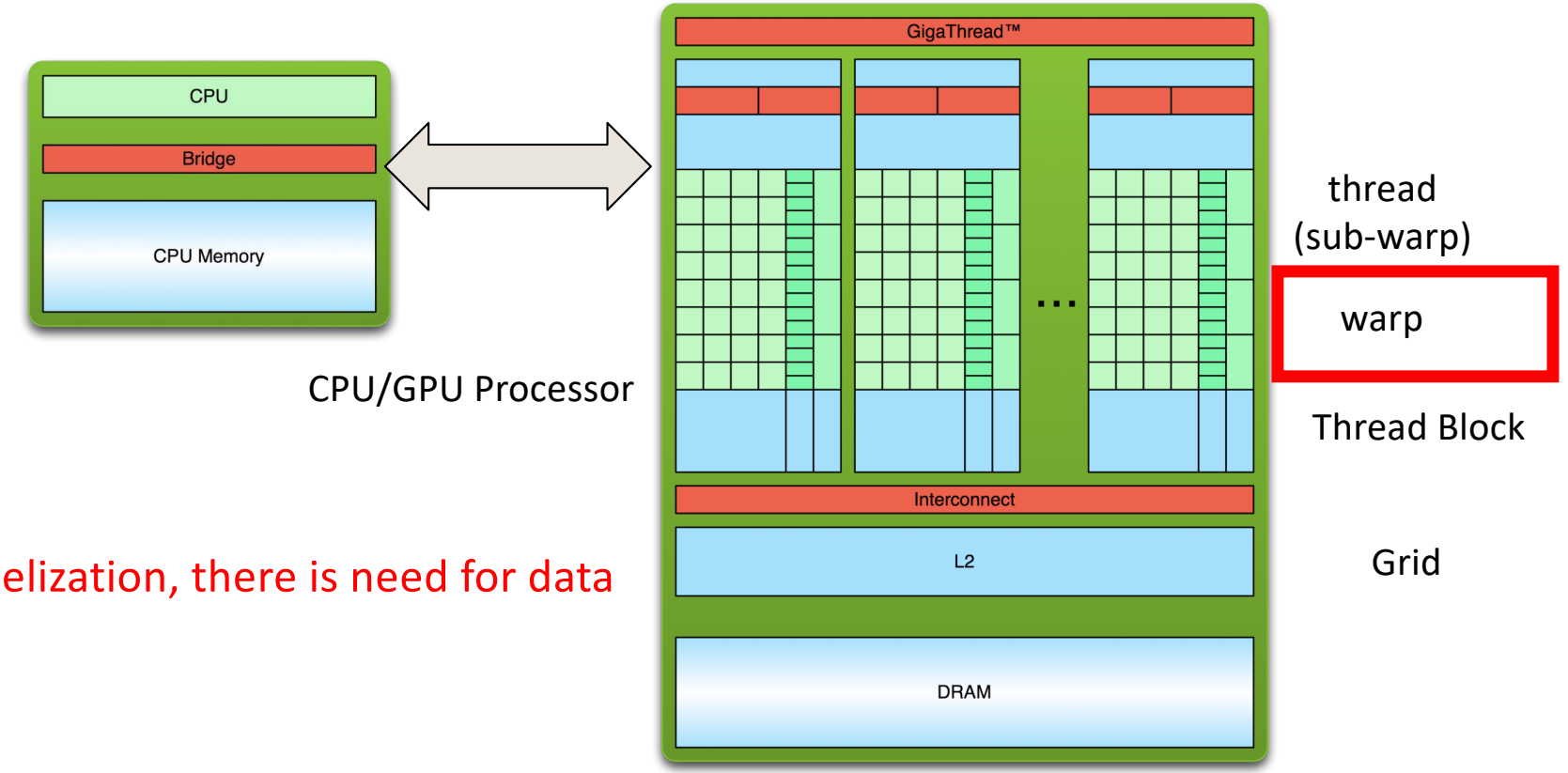
```
__global__kernel() {
    extern __shared__ data[];
    if (threadIdx.x == 0) {
       // write data to shared memory
    }
    __syncthreads();
    if (threadIdx.x == 1) {
       // read data from shared memory
    }

}
```



Warp-level data sharing

UNIVERSITY OF OREGON | Computer Science

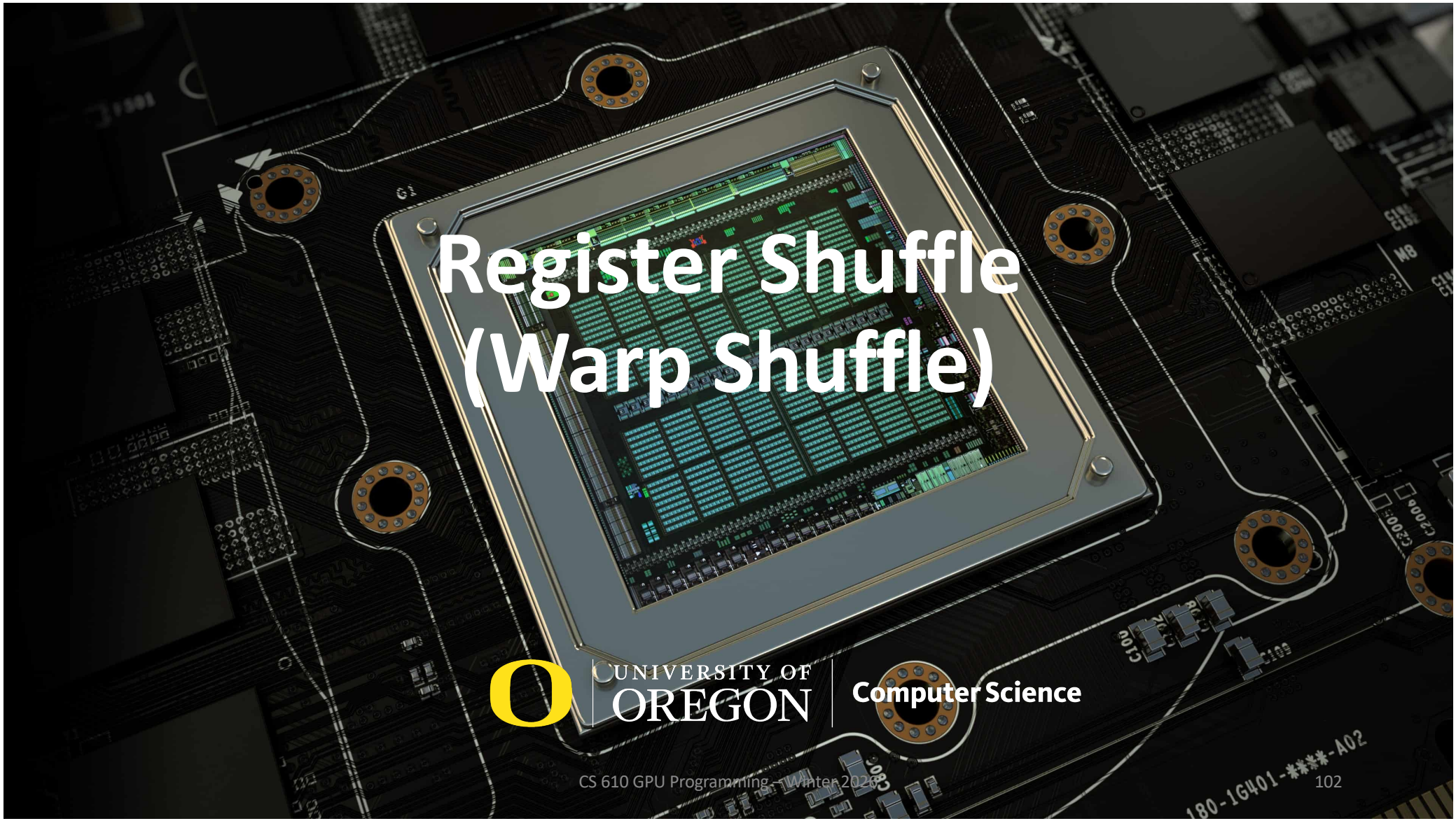
Parallelisms



If there is parallelization, there is need for data sharing

Warp-level data sharing

- Sharing data between threads in a warp
- Two common accessible memory (same as block-level data sharing)
  - Shared memory: faster, less space (65K-225K)
  - Global memory: slower, larger space
- Most efficient approach: direct register access
  - Access other thread's registers directly
  - Enabled with register shuffle instructions



Register Shuffle (Warp Shuffle)

UNIVERSITY OF OREGON | Computer Science

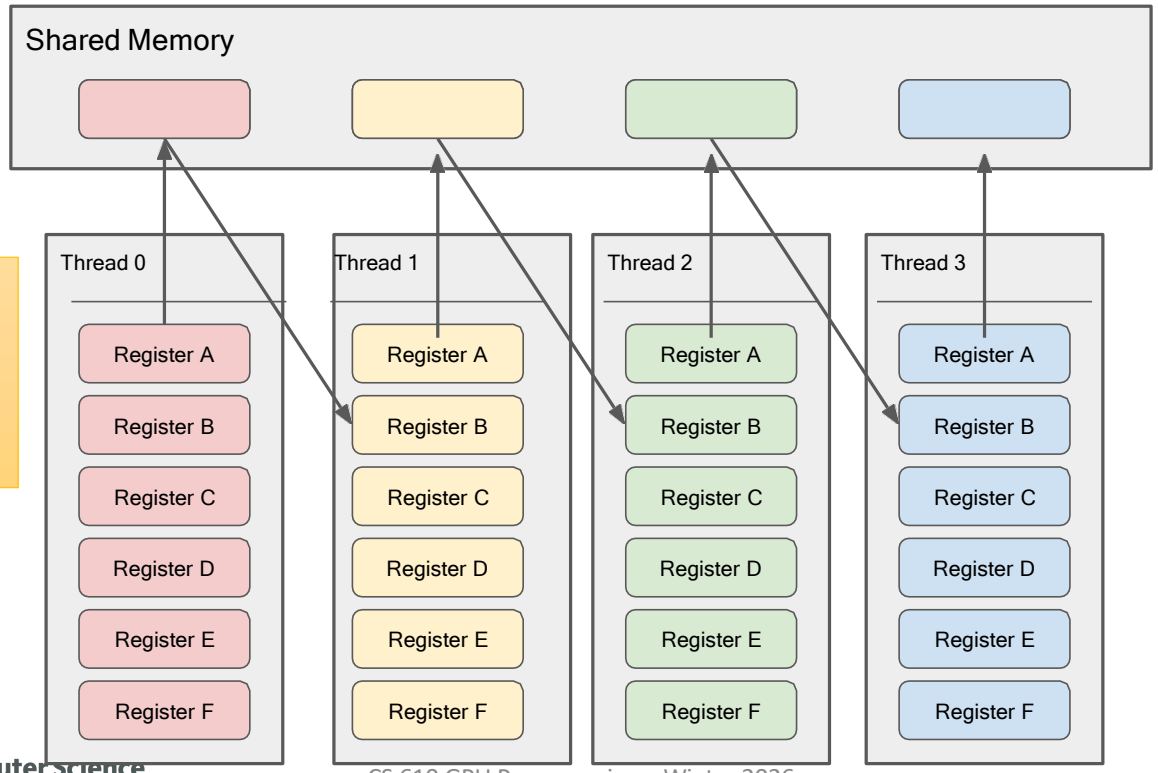
Warp Shuffle

- For moving/comparing data between threads in a block it is possible to use Shared Memory
- For moving/comparing data between threads in a warp (known as lanes in this context) it is possible to use a *warp shuffle*
  - Direct exchange of information between two threads
  - Does not require shared memory
  - Implicit synchronization (no need for `__syncthreads`)
  - Works by allowing threads to read another threads registers
  - Only threads within the same warp can share registers
  - https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#warp-shuffle-functions

Data Sharing via Shared Memory

```
extern __shared___ data[];
// register to shared memory
__syncthreads();
// shared memory to register
```

The latency associated with sharing a value with a neighboring thread is twice the shared memory latency (40-60 cycles)

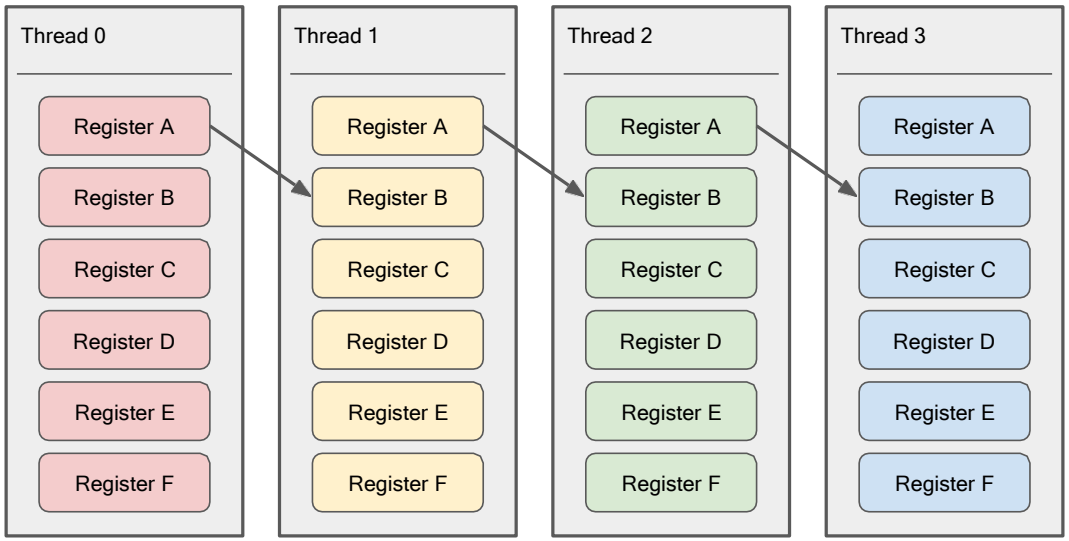


Data Sharing via Warp Shuffle (Register Sharing)

```
int x = threadIdx.x;
int y = __shfl_up_sync(0xffffffff, x, 1, warpSize);
```

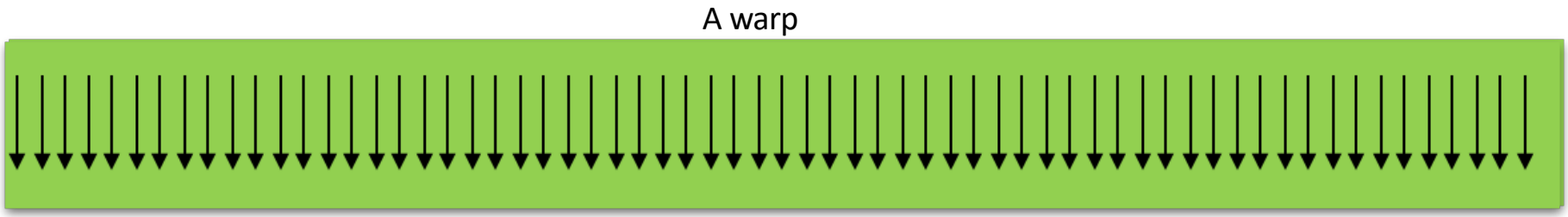
```
// stored in Register A
// stored in Register B
```

The latency associated with sharing a value with a neighboring thread is < 10 cycles



Review: Warp and Lane

- Each thread in a warp is also called a lane
- Lane index: 0 … 31
- Calculating warp id and lane id
  - Warp ID = Thread ID / 32
  - Lane ID = Thread ID % 32
- Faster way
  - Warp ID = Thread ID >> 5
  - Lane ID = Thread ID & 0x1f

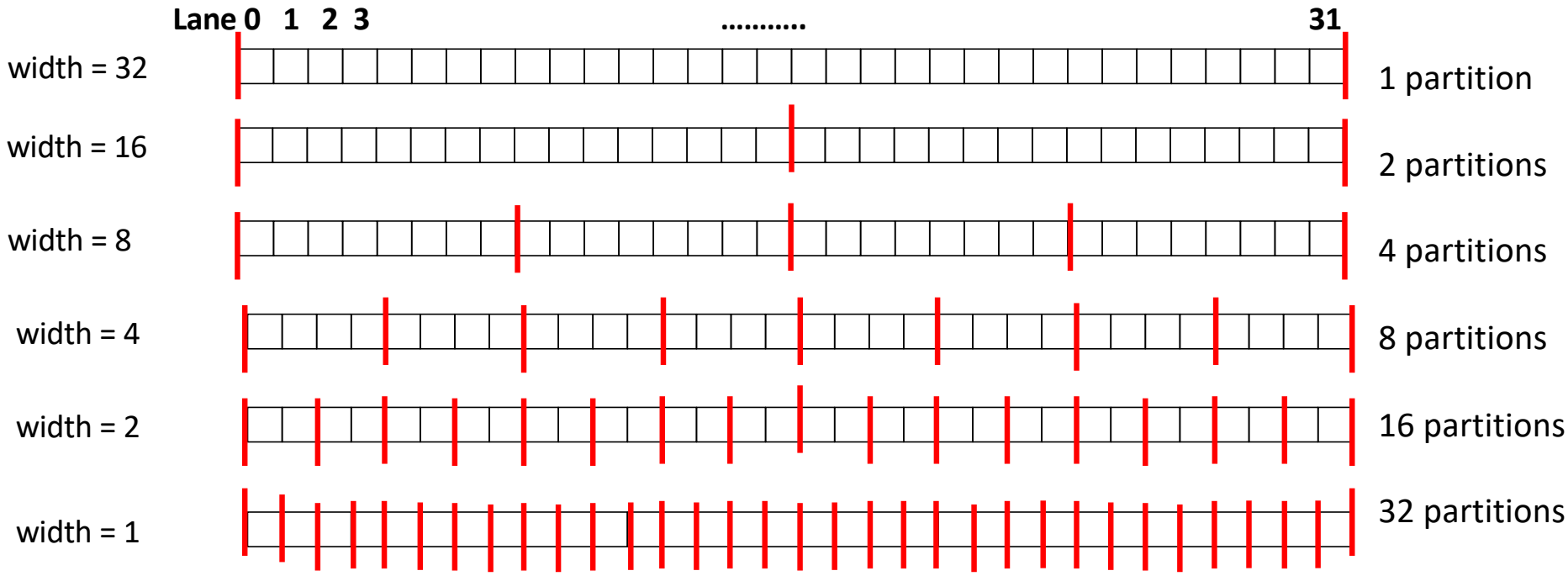


Direct Copy

```
T __shfl_sync(unsigned mask, T var, int srcLane, int width = warpSize);
```

- Returns the value of var held by the thread whose ID is given by srcLane
- mask: indicate which lane is participating (each bit per lane, 1=participate; 0= not participate)
- var: register to be shared with others
- srcLane: source lane to get the data
- width: size of partition. Must be power of two. Each partition works independently

Partition


Lane 0 1 2 3 31
width = 32
1 partition
width = 16
2 partitions
width = 8
4 partitions
width = 4
8 partitions
width = 2
16 partitions
width = 1
32 partitions

Mask

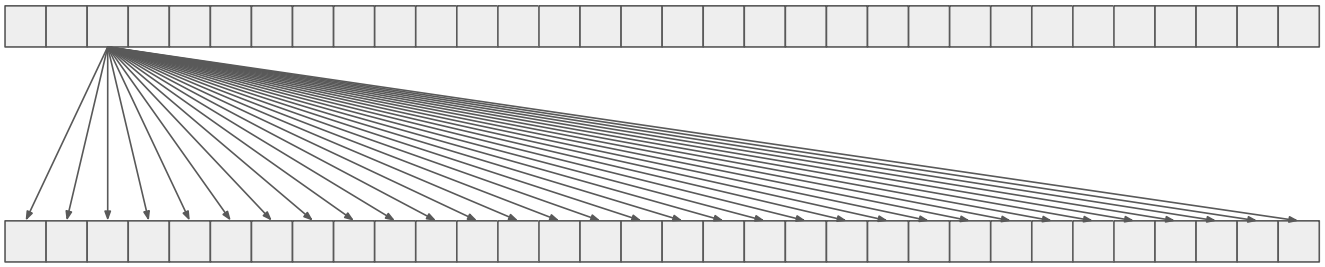
```
0xffffffff = 111 … 1 (32 bit 1s)
All 32 lanes participate
```

Direct Copy

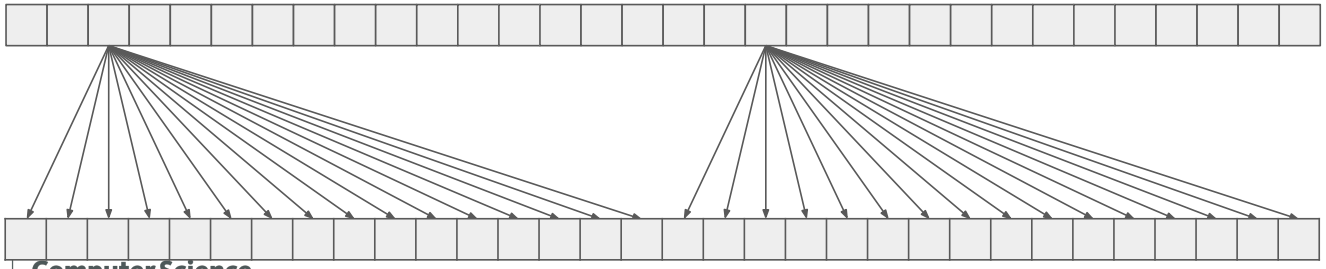
What happens when src is same for all threads?
What happens when src is different for different threads?

```
T __shfl_sync(unsigned mask, T var, int srcLane, int width = warpSize);
```

```
int y = __shfl_sync(0xffffffff, x, 2);    // copy from lane 2 to rest lanes in the warp
```



```
int y = __shfl_sync(0xffffffff, x, 2, 16); // copy from lane 2 and 18 to rest lanes in each half-warps
```



Shuffle Up

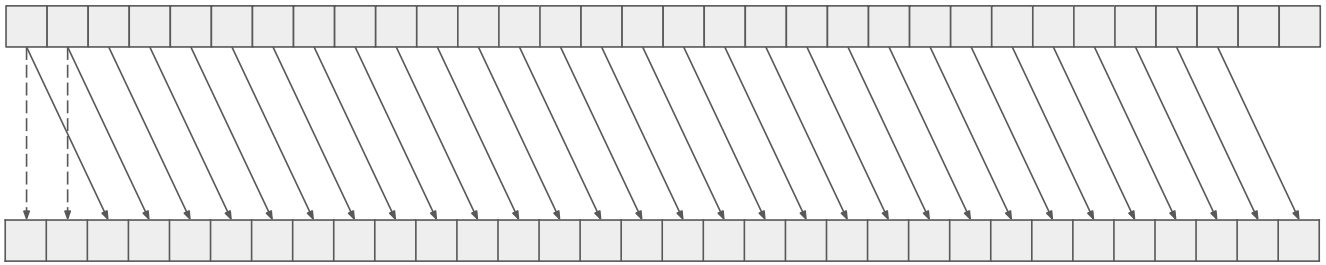
```
T __shfl_up_sync(unsigned mask, T var, unsigned int delta, int width = warpSize);
```

- Returns the value of var held by the thread whose ID is given by srcLane
- mask: indicate which lane is participating (each bit per lane, 1=participate; 0= not participate)
- var: register to be shared with others
- delta: source lane = current lane – delta (copy from a lower lane)
- width: size of partition. Must be power of two. Each partition works independently

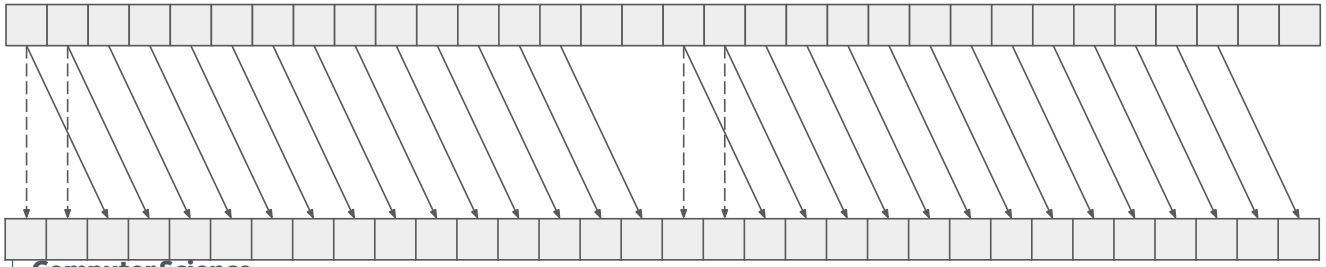
Shuffle Up

```
T __shfl_up_sync(unsigned mask, T var, unsigned int delta, int width = warpSize);
```

```
int y = __shfl_up_sync(0xffffffff, x, 2);  // shift values two lanes to the right within a warp
```



```
int y = __shfl_up_sync(0xffffffff, x, 2, 16); // shift values two lanes to the right within a half-warp
```



Shuffle Down

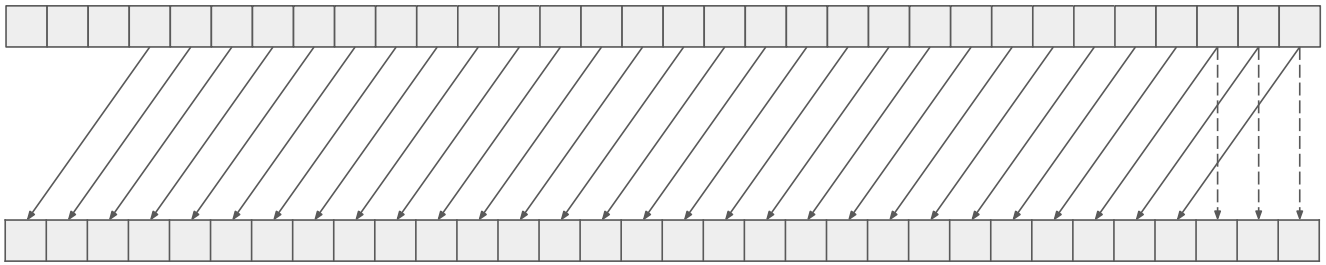
```
T __shfl_down_sync(unsigned mask, T var, unsigned int delta, int width = warpSize);
```

- Returns the value of var held by the thread whose ID is given by srcLane
- mask: indicate which lane is participating (each bit per lane, 1=participate; 0= not participate)
- var: register to be shared with others
- delta: source lane = current lane + delta (copy from a higher lane)
- width: size of partition. Must be power of two. Each partition works independently

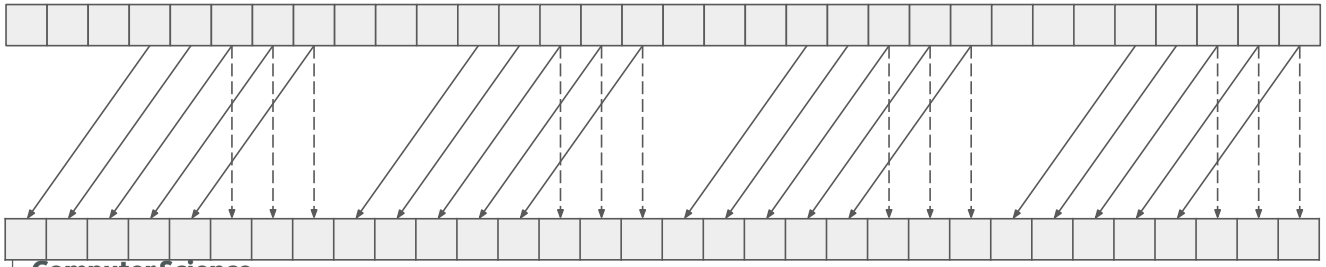
Shuffle Down

```
T __shfl_down_sync(unsigned mask, T var, unsigned int delta, int width = warpSize);
```

```
int y = __shfl_down_sync(0xffffffff, x, 3); // shift values three lanes to the right within a warp
```



```
int y = __shfl_down_sync(0xffffffff, x, 3, 8);  // shift values three lanes to the right within a quarter-warp
```



Butterfly Exchange

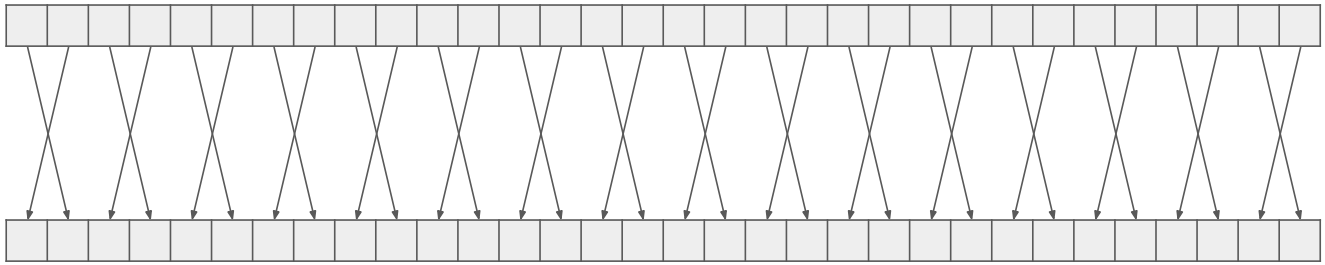
```
T __shfl_xor_sync(unsigned mask, T var, int laneMask, int width = warpSize);
```

- Returns the value of var held by the thread whose ID is given by srcLane
- mask: indicate which lane is participating (each bit per lane, 1=participate; 0= not participate)
- var: register to be shared with others
- delta: source lane = current lane XOR laneMask
- width: size of partition. Must be power of two. Each partition works independently

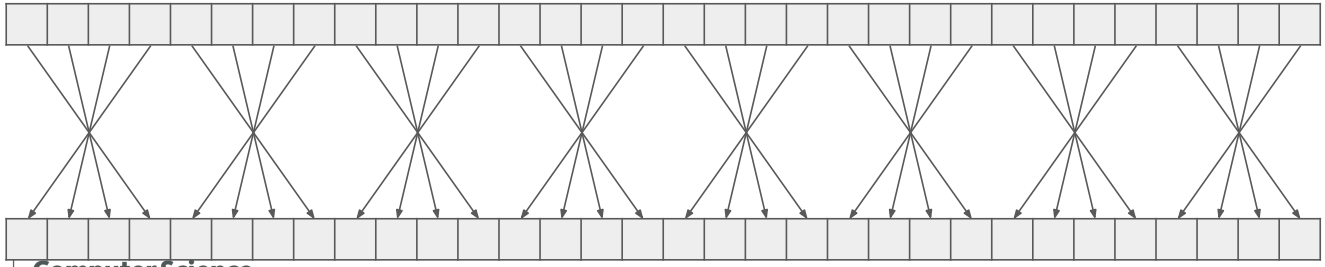
Butterfly Exchange

```
T __shfl_xor_sync(unsigned mask, T var, int laneMask, int width = warpSize);
```

```
int y = __shfl_xor_mask(0xffffffff, x, 1); // trade values with a neighbor
```



```
int y = __shfl_xor_sync(0xffffffff, x, 3); // invert order of groups of four elements
```



Shuffle function arguments

- `T __shfl_sync(unsigned mask, T var, int srcLane, int width=warpSize);`
- `T __shfl_up_sync(unsigned mask, T var, unsigned int delta, int width=warpSize);`
- `T __shfl_down_sync(unsigned mask, T var, unsigned int delta, int width=warpSize);`
  - delta is the n step used for shuffling
- `T __shfl_xor_sync(unsigned mask, T var, int laneMask, int width=warpSize);`
  - Source lane determined by bitwise XOR with `laneMask`

- Mask is a bit mask for the warp to indicate which threads participate
- T can be `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `float` or `double`
- Optional width argument
  - Must be a power of 2 and less than or equal to warp size
  - If smaller than warp size each subsection acts independently (own wrapping)

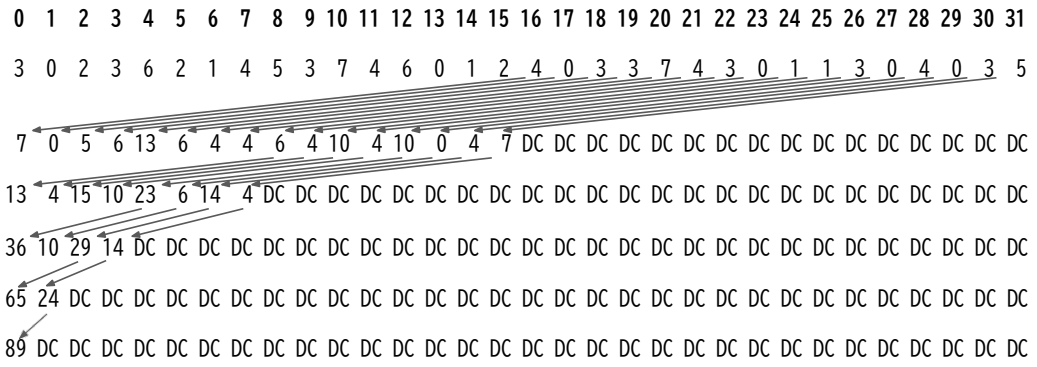
Warp-level Reduction

```
__device__ __forceinline__ int warpReduceSum(int sum) {
    sum += __shfl_down_sync(0xffffffff, sum, 16);
    sum += __shfl_down_sync(0xffffffff, sum, 8);
    sum += __shfl_down_sync(0xffffffff, sum, 4);
    sum += __shfl_down_sync(0xffffffff, sum, 2);
    sum += __shfl_down_sync(0xffffffff, sum, 1);
    return sum;
}
```

Only the thread in lane 0 has a meaningful result!

Width is omitted (default = 32)

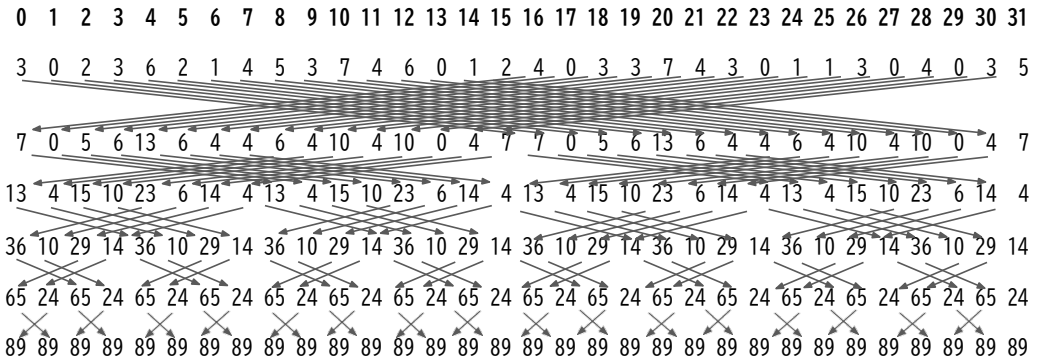
How to find the maximum or minimum?



Warp-level Reduction

```
__device__ forceinline__ int warpReduceSum(int sum) {
    sum += __shfl_xor_sync(0xffffffff, sum, 16);      // 0-16, 1-17, 2-18, etc.
    sum += __shfl_xor_sync(0xffffffff, sum, 8);// 0-8, 1-9, 2-10, etc.
    sum += __shfl_xor_sync(0xffffffff, sum, 4);// 0-4, 1-5, 2-6, etc.
    sum += __shfl_xor_sync(0xffffffff, sum, 2);// 0-2, 1-3, 4-6, 5-7, etc.
    sum += __shfl_xor_sync(0xffffffff, sum, 1);// 0-1, 2-3, 4-5, etc.
    return sum;
}
```

All threads return the sum



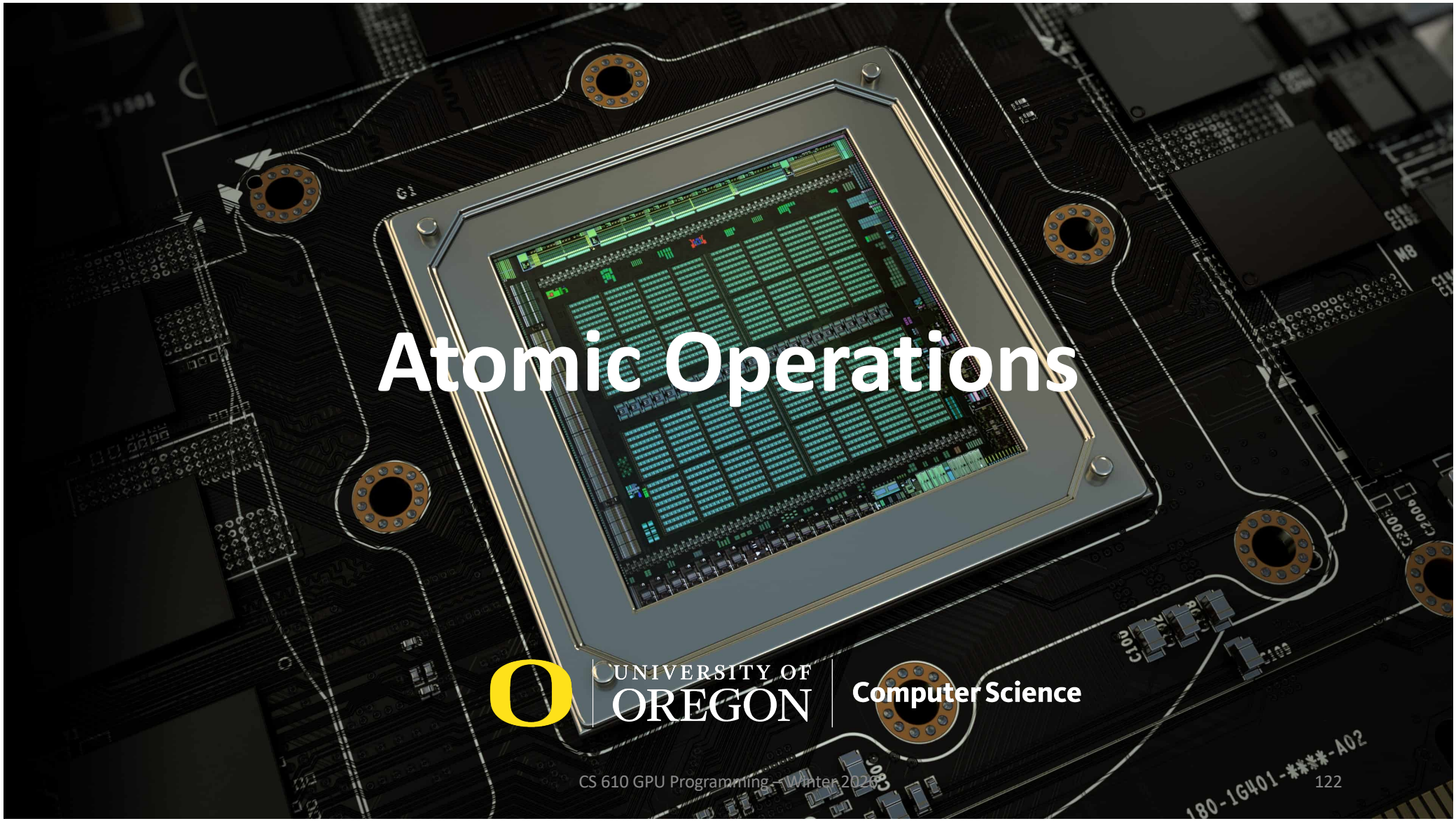
Warp Voting

- Warp shuffles allow data to be exchanged between threads in a warp
- Warp voting allows threads to test a condition across all threads in a warp
  - `int __all_sync(unsigned mask, int predicate)`
    - True if the condition is met by all threads in the warp
  - `int __any_sync(unsigned mask, int predicate)`
    - True is any thread in warp meets condition
  - `unsigned int __ballot_sync(unsigned mask, int predicate)`
    - Sets the n^{th} bit of the return value based on the n^{th} threads condition value
- All warp voting functions are single instruction and act as sync barrier
  - Only active threads participate, does not block like `syncthreads()`
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#warp-vote-functions

Warp Voting Example

```
__global__void voteAllKernel(unsigned int *input, unsigned int *result)
{
  int i = threadIdx.x + blockIdx.x*blockDim.x;
  int j = i % WARP_SIZE;

  int vote_result = __all_sync(0xffffffff, input[i]);

  if (j==0)
    result[i / WARP_SIZE] = vote_result;
```

- For each first thread in the warp calculate if all threads in the warp have `true` valued input
- Save the warp vote to a compact array
  - A reduction of factor 32



Atomic Operations

UNIVERSITY OF OREGON | Computer Science

What is wrong with the following

```
__global__ void max_kernel(int *a)
{
  __shared__ int max;

  int my_local = a[threadIdx.x + blockIdx.x*blockDim.x];

  if (my_local > max)
    max = my_local;
}
```

- More than one thread may try to modify max at the same time
  - Race condition

Each thread does:
1. read max
2. compare max
3. update max

```
__global__ void max_kernel(int *a)
{
  __shared__ int max;

  int my_local = a[threadIdx.x + blockIdx.x*blockDim.x];

  if (my_local > max)
    max = my_local;
}
```

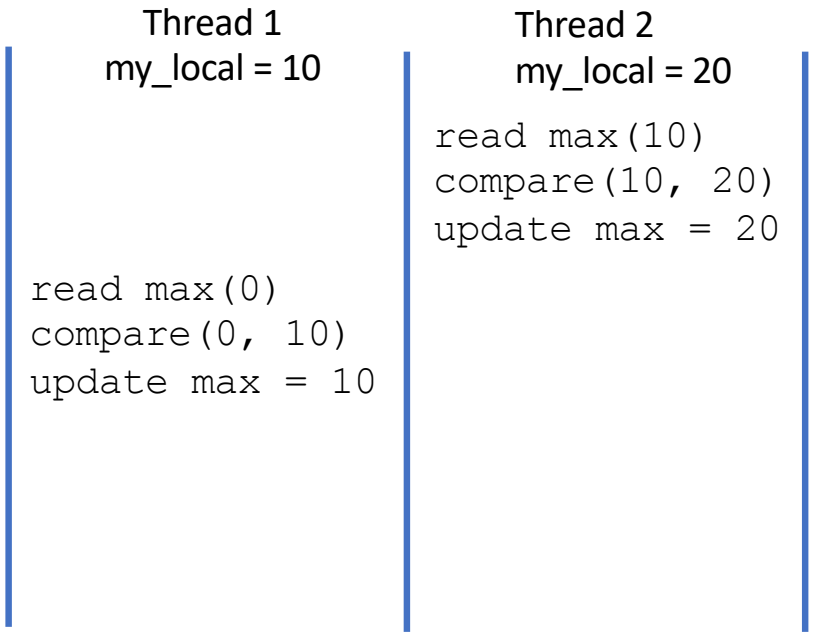
__shared__ max = 0

Thread 1 my_local = 10	Thread 2 my_local = 20

`read max(0)`
`compare(0, 10)`
`update max = 10`	

	`read max(10)`
	`compare(10, 20)`
	`update max = 20`

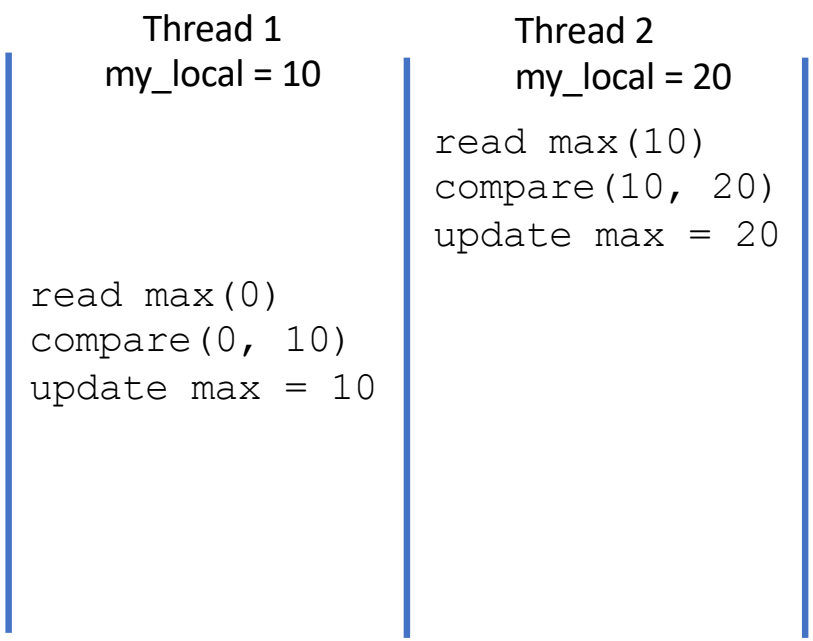
__shared__ max = 0


Thread 1
my_local = 10
read max(0)
compare(0, 10)
update max = 10
Thread 2
my_local = 20
read max(10)
compare(10, 20)
update max = 20

__shared__ max = 0

Thread 1 my_local = 10	Thread 2 my_local = 20
`read max(0)`	`read max(0)`
`compare(0, 10)`	`compare(0, 20)`
`update max = 10`	`update max = 20`

__shared__ max = 0



Need to guarantee execution without interruption

Atomics

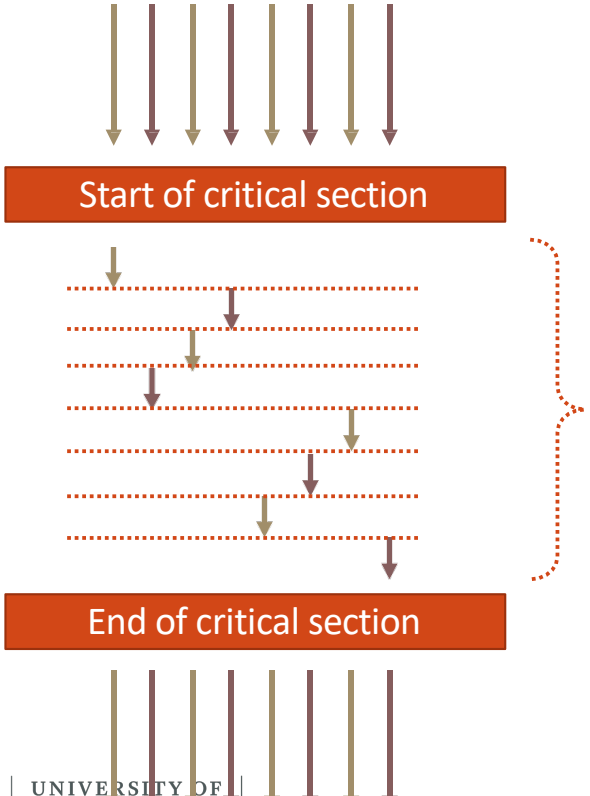
- Atomics are used to ensure correctness when concurrently reading and writing to a memory location (global or shared)
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#atomic-functions

No need for assignment to a variable when using the atomic functions

```
__global__ void max_kernel(int *a)
{
  __shared__ int max;

  int my_local = a[threadIdx.x + blockIdx.x*blockDim.x];

  atomicMax(&max, my_local);

}
```

- No race condition
- Function supported in *most* hardware
  - Some older generation GPUs lack shared memory and floating point atomic etc.

Serialization



- What happens when using atomics?

  Execution is serialized for each thread accessing the shared value

atomicMax

```
T atomicMax(T * address, T val)
```

- Read the existing data at address in global or shared memory
- Compute the **maximum** between existing data and val
- Store the **result** back to memory at the same address
- Return the existing value
- All above steps are done atomically
- T can be int, unsigned int, long long, unsigned long long

All atomic functions

```
T atomicMax(T * address, T val)
T atomicMin(T * address, T val)
T atomicAdd(T * address, T val)
T atomicSub(T * address, T val)
T atomicExch(T * address, T val)
T atomicInc(T * address, T val)
T atomicDec(T * address, T val)
T atomicAnd(T * address, T val)
T atomicOr(T * address, T val)
T atomicXor(T * address, T val)
```

Variants for different scopes

Thread block scope:

```
T atomicMax_block(T * address, T val)
```

Device/Grid scope:

```
T atomicMax(T * address, T val)
```

System scope:

```
T atomicMax_system(T * address, T val)
```

This applies to all atomic functions

Special Compare And Swap atomic function

```
T atomicCAS(T* address, T compare, T val)
```

- Read the existing data at address in global or shared memory
- Compute (existing == compare ? val : existing)
- Store the **result** back to memory at the same address
- Return the existing value
- All above steps are done atomically
- T can be int, unsigned int, long long, unsigned long long

Special Compare And Swap atomic function

```
T atomicCAS(T* address, T compare, T val)
```

What does it do?

We try to guess the existing value at address is compare

- If we guess right (i.e., existing == compare)
  - We gain the right to update existing value to be `val`
- If we guess wrong (i.e., existing != compare)
  - We do NOT have the right to update existing value
  - Instead, we are returned with *up-to-date* existing value
  - We can use the *up-to-date* existing value to guess later on
  - However, there is no guarantee our version of existing value is always up-to-date
  - So, we need to keep trying

Application of atomicCAS function

- How can we implement critical sections?

```
__device__int lock = 0;

__global__void kernel() {
  bool need_lock = true;
  // get lock
  while (need_lock) {
    if (atomicCAS(&lock, 0, 1)==0) {
      //critical code section
      atomicExch(&lock, 0);
      need_lock = false;
    }
  }
}
```

```
int atomicCAS(int* address, int compare, int val)
```

Performs the following in a single atomic transaction (atomic instruction)

```
*address=(*address==compare)? val : *address;
```

Returning the old value at the address

Application of atomicCAS function

- Most atomic operation only support integer types: int, unsigned int, long long, unsigned long long
- How to implement floating point atomic operation?
- Type cast: https://docs.nvidia.com/cuda/cuda-math-api/group__CUDA__MATH__INTRINSIC__CAST.html

```
__device__ double atomicAdd(double* address, double val)
{
    unsigned long long int* converted_address = (unsigned long long int*)address;
    unsigned long long int existing_val = * converted_address
    unsigned long long int guessed_val;
    do {
        guessed_val = existing_val;
        desired_val = __double_as_longlong(val + __longlong_as_double(guessed_val));
        old = atomicCAS(converted_address, guessed, desired_val);
    } while (guessed_val != existing_val);
return __longlong_as_double(existing_val);
}
```

Application of atomicCAS function

- Most atomic operation only support integer types: int, unsigned int, long long, unsigned long long
- How to implement floating point atomic operation?
- Type cast: https://docs.nvidia.com/cuda/cuda-math-api/group__CUDA__MATH__INTRINSIC__CAST.html

```
__device__ double atomicMax(double* address, double val)
{
    unsigned long long int* converted_address = (unsigned long long int*)address;
    unsigned long long int existing_val = * converted_address
    unsigned long long int guessed_val;
    do {
        guessed_val = existing_val;
        desired_val = __double_as_longlong(max(val, __longlong_as_double(guessed_val)));
        old = atomicCAS(converted_address, guessed, desired_val);
    } while (guessed_val != existing_val);
return __longlong_as_double(existing_val);
}
```

Four-Level Parallelization Hierarchy